



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

EXTENSIÓN EXPERIMENTAL DE APPLICATIVO EN GHC MEDIANTE
REORDENAMIENTO DE MÓNADAS CONMUTATIVAS EN PROGRAMAS
PROBABILÍSTICOS DE HASKELL

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

ESTEBAN ROMERO BERRÍOS

PROFESOR GUÍA:
FEDERICO OLMEDO B.

PROFESOR CO-GUÍA:
ISMAEL FIGUEROA P.

MIEMBROS DE LA COMISIÓN:
GONZALO NAVARRO B.
IGNACIO SLATER M.

SANTIAGO DE CHILE
2026

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Una dedicatoria corta.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Tabla de Contenido

1. Introducción	1
1.1. Contexto	1
1.2. Problema y Enfoque de Solución	3
1.3. Objetivos	6
1.4. Contribuciones	6
2. Marco Teórico	7
2.1. Fundamentos de Haskell	7
2.2. Abstracciones Funcionales	9
2.2.1. Functores	9
2.2.2. Aplicativos	10
2.2.3. Mónadas	12
2.3. Do-Notation	14
2.4. Mónadas Conmutativas	15
2.5. Mónadas de Probabilidad	16
2.6. ApplicativeDo	19
2.6.1. Etapa de Rearrange	19
2.6.2. Etapa de Desugaring	21
2.6.3. Aplicación del algoritmo	23
3. Problema	26
3.1. Limitación del Orden Sintáctico de ApplicativeDo	26

3.2. Evidencia del Problema y Oportunidad	27
3.3. Formulación	29
4. Solución	31
4.1. Enfoque Propuesto	31
4.2. Diseño Algorítmico	32
4.2.1. Modelo de Dependencias	33
4.2.2. Construcción del Grafo de Precedencia	35
4.2.3. Enumeración de Reordenamientos Válidos	36
4.2.4. Evaluación y Selección de Candidatos	39
4.3. Implementación en GHC	39
4.3.1. Ubicación en el Pipeline del Compilador	40
4.3.2. Activación Declarativa mediante QualifiedDo	41
4.3.3. Representación de Statements y Construcción del Grafo	41
4.3.4. Generación de Permutaciones	43
4.3.5. Integración con ApplicativeDo	44
4.3.6. Selección Forzada para Validación	45
5. Validación	47
5.1. Metodología	47
5.2. Artefactos de Validación	48
5.3. Métricas	48
5.4. Corpus	49
5.4.1. Corpus Sintético Maybe	49
5.4.2. Corpus Sintético Probabilístico	50
5.4.3. Control IO	50
5.5. Resultados	51
5.6. Amenazas a la Validez	52

6. Conclusión	53
6.1. Resumen del Trabajo	53
6.2. Cumplimiento de Objetivos	53
6.3. Limitaciones	54
6.4. Trabajo Futuro	55
6.5. Cierre	55
Bibliografía	57
Apéndice A. Anexos Técnicos	58
A.1. Reproducibilidad	58
A.2. Pipeline de Validación	58
A.3. Logs y Trazas	58
A.4. Corpus	59
A.5. Detalles de Implementación	59

Índice de Tablas

1.1. Probabilidad de aprobar el examen final según preparación y dificultad. . . .	3
1.2. Comparación de planes y costos estructurales para el ejemplo académico. . .	5
4.1. Aplicación del algoritmo de Kahn al grafo del ejemplo probabilístico.	37
4.2. Reordenamientos topológicos válidos para el ejemplo probabilístico.	39

Índice de Ilustraciones

1.1.	Grafo de precedencia del Código 1.1 dispuesto según el orden original [1,2,3,4].	4
1.2.	Mismo grafo de precedencia de la Figura 1.1 dispuesto según el reordenamiento de statements [1,3,2,4].	4
2.1.	Etapa de <i>rearrange</i> propuesta por Marlow et al. [19, p. 95].	20
2.2.	Etapa de <i>desugaring</i> propuesta por Marlow et al. [19, p. 96].	22
4.1.	Pipeline conceptual de la extensión propuesta.	32
4.2.	Árbol de llamadas para enumerar los reordenamientos topológicos del ejemplo probabilístico. La raíz representa la invocación inicial y los otros 15 estados corresponden a llamadas recursivas; las cinco hojas resaltadas son los candidatos obtenidos.	38

Capítulo 1

Introducción

1.1. Contexto

La programación probabilística es un paradigma que permite describir modelos con incertidumbre mediante programas que incorporan operaciones para extraer valores desde distribuciones de probabilidad y, en muchos lenguajes, condicionar esos valores mediante observaciones. A diferencia de un programa determinista, cuyo resultado queda determinado por su entrada, un programa probabilístico especifica implícitamente una distribución sobre sus posibles resultados. La inferencia probabilística consiste en obtener una representación de esa distribución o calcular propiedades de ella, como probabilidades, valores esperados o muestras [11].

Como lenguaje de modelado, la programación probabilística se ha aplicado en dominios como visión computacional, teoría de códigos, protocolos criptográficos, biología y análisis de confiabilidad [11]. Su relación con el aprendizaje automático y la inteligencia artificial es especialmente estrecha: los modelos probabilísticos permiten representar y manipular incertidumbre sobre datos, modelos y predicciones, mientras que los sistemas de programación probabilística permiten expresar esos modelos como procesos computacionales sobre los cuales aplicar inferencia bayesiana [6, 24].

El resultado observado al evaluar un programa probabilístico depende de la estrategia de interpretación utilizada. Bajo una interpretación por muestreo, cada ejecución concreta produce una muestra de la distribución inducida; por ejemplo, una ejecución del lanzamiento de una moneda retorna “cara” o “sello”. Una estrategia de inferencia exacta, en cambio, puede construir una representación explícita de la distribución completa. En ambos casos, el programa describe semánticamente la distribución sobre todos los resultados posibles, no solo una muestra particular [11, 3].

En lenguajes funcionales puros como **Haskell**, es posible encapsular este tipo de cálculos en tipos de datos y componerlos mediante mónadas de probabilidad. Para distribuciones discretas y finitas, una representación monádica puede construir explícitamente los resultados posibles junto con sus probabilidades, lo que permite realizar inferencia exacta sobre la distribución inducida por el programa [3]. El mismo enfoque monádico también puede servir

como interfaz para métodos de inferencia más generales, sin restringirse a la enumeración exhaustiva [24].

Estas representaciones permiten realizar consultas y operaciones sobre la distribución de salida de un programa. Por ejemplo, la distribución conjunta asociada al lanzamiento de una moneda seguido del lanzamiento de un dado puede expresarse mediante la *do-notation* de Haskell [3]:

```
flipCoinRollDice = do
  c <- uniform [Head, Tail]
  d <- uniform [1 .. 6]
  return (c, d)
```

En cada resultado posible del código anterior, la variable `c` contiene un resultado del lanzamiento de la moneda y la variable `d` contiene una cara del dado. Como la segunda elección no depende del resultado de la primera, `return (c, d)` construye la distribución conjunta de ambos experimentos. Su soporte contiene los doce pares obtenidos al combinar los dos resultados de la moneda con las seis caras del dado y, bajo las elecciones uniformes del ejemplo, cada par tiene probabilidad $1/12$ [3].

Tal como se aprecia en el ejemplo, la *do-notation* es especialmente conveniente para programas probabilísticos: un statement de enlace puede introducir una variable aleatoria o un resultado intermedio, y los statements posteriores pueden depender de los valores ya definidos. En el desazucarado monádico convencional de Haskell, un statement de la forma `p <- e` se traduce mediante el operador *bind* (`>>=`); los statements sin enlace emplean (`>>`) y las declaraciones `let` poseen una regla propia [16, sección 3.14]. El operador *bind* permite que la forma de un cómputo posterior dependa del valor producido por uno anterior, lo que resulta fundamental para modelar efectos como estado, entrada/salida, fallas, no determinismo o distribuciones probabilísticas [26, 3].

La secuencia monádica es correcta para el caso general, pero puede ocultar independencia entre subcómputos. Esta tensión fue abordada en GHC mediante la extensión **ApplicativeDo**, propuesta por Marlow et al. [19]. La extensión analiza las dependencias de un bloque escrito con *do-notation* y, cuando es posible, construye un plan de ejecución mixto que utiliza operadores aplicativos como (`<$>`) y (`<*>`) junto con las operaciones monádicas que todavía sean necesarias [19, 9]. De este modo, el programador conserva la sintaxis habitual y el plan resultante puede exponer independencia que ciertas instancias aplicativos aprovechan mediante paralelismo o agrupación de operaciones; la transformación, por sí sola, no garantiza una ejecución multihilo.

Sin embargo, el algoritmo estándar de **ApplicativeDo** está diseñado para mónadas generales, sin asumir que sus efectos sean conmutativos. Por esta razón, para obtener el nuevo plan de ejecución agrupa statements compatibles pero no cambia su orden relativo: una permutación de estos podría alterar efectos observables, mientras que el intercambio de cómputos independientes solo es válido bajo una hipótesis adicional de conmutatividad [19, pp. 94–95]. La subsección siguiente muestra, mediante un programa probabilístico concreto, cómo esta restricción puede impedir que el algoritmo de **ApplicativeDo** genere un plan de ejecución

de menor costo.

1.2. Problema y Enfoque de Solución

Considérese un modelo probabilístico sencillo sobre el desempeño de un estudiante. El modelo representa el nivel de preparación del estudiante, el resultado de un quiz, la dificultad de un examen final y el resultado de ese examen. La preparación y la dificultad se modelan como elecciones equiprobables e independientes. En cambio, los resultados de las evaluaciones se modelan mediante distribuciones condicionadas: la probabilidad de aprobar el quiz es $1/4$ para una preparación baja y $3/4$ para una preparación alta; la probabilidad de aprobar el examen final se resume en la Tabla 1.1.

Preparación	Dificultad	Probabilidad de aprobar
Low	Easy	$1/2$
Low	Hard	$1/4$
High	Easy	$3/4$
High	Hard	$1/2$

Tabla 1.1: Probabilidad de aprobar el examen final según preparación y dificultad.

Las distribuciones condicionales se implementan mediante las funciones `quizGiven` y `examGiven`. El cómputo conjunto se escribe con la *do-notation* de la siguiente forma:

```
studentResults = do
  preparation <- Dist.uniform [Low, High]
  quiz <- quizGiven preparation
  difficulty <- Dist.uniform [Easy, Hard]
  finalExam <- examGiven preparation difficulty
  return (preparation, quiz, difficulty, finalExam)
```

Código 1.1: Modelo probabilístico del desempeño de un estudiante.

Para analizar el bloque, se numeran los cuatro statements previos al `return` como `s1`, `s2`, `s3` y `s4`. El statement `s2` usa la preparación definida por `s1`; el statement `s4` usa tanto la preparación de `s1` como la dificultad definida por `s3`. No existe, en cambio, una dependencia entre el resultado del quiz en `s2` y la dificultad en `s3`. Las precedencias necesarias son, por tanto, $s1 \rightarrow s2$, $s1 \rightarrow s4$ y $s3 \rightarrow s4$.

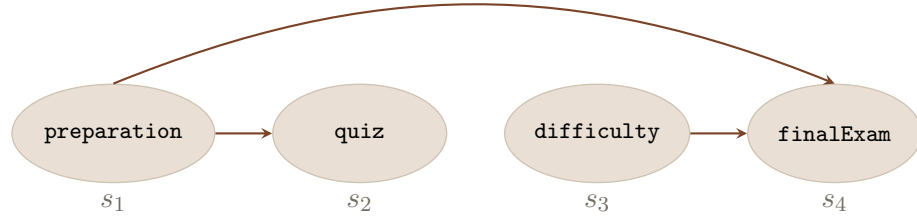


Figura 1.1: Grafo de precedencia del Código 1.1 dispuesto según el orden original [1,2,3,4].

En el anterior grafo de precedencia, la posición horizontal de los nodos representa el orden de los statements: un nodo ubicado a la izquierda aparece antes que uno ubicado a la derecha. Una disposición respeta las dependencias del programa solo si todas las aristas avanzan de izquierda a derecha, pues cada arista exige que el statement que produce un valor aparezca antes del statement que lo usa. Por consiguiente, una arista dirigida hacia la izquierda indicaría que ese orden invierte y rompe la dependencia.

El grafo no impone un orden relativo entre s_2 y s_3 , de modo que pueden intercambiarse para obtener el orden [1,3,2,4]. Este orden conserva s_1 antes de s_2 y s_4 , y s_3 antes de s_4 ; por ello, las tres aristas siguen avanzando hacia la derecha, como muestra la Figura 1.2. El respeto de estas precedencias establece la validez de un reordenamiento de statements, mientras que la hipótesis de conmutatividad permite afirmar que es semánticamente equivalente al orden original.

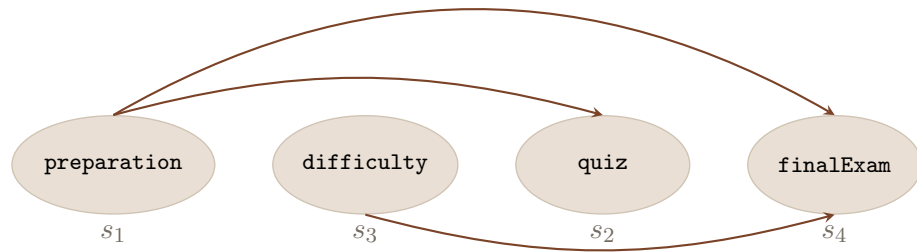


Figura 1.2: Mismo grafo de precedencia de la Figura 1.1 dispuesto según el reordenamiento de statements [1,3,2,4].

Una vez fijado un orden válido de los statements, todavía debe determinarse cómo se combinan sus cálculos. En esta memoria se denomina *plan de ejecución* a una representación abstracta de esa composición: el plan indica qué partes permanecen secuenciadas y cuáles pueden agruparse aplicativamente cuando las dependencias lo permiten. No describe una planificación concreta de hilos ni tiempos reales de ejecución, sino la estructura del cómputo sobre la que se comparan las distintas alternativas.

Para escribir los planes de ejecución del programa del Código 1.1 se utilizan dos operadores conceptuales. El símbolo $;$ indica composición secuencial: el plan de la derecha comienza cuando finaliza el de la izquierda. El símbolo $|$ agrupa aplicativamente planes independientes y expresa paralelismo potencial, no necesariamente una ejecución multihilo.

Sobre esta notación se adopta el siguiente modelo de costos estructural simplificado. Sea C la función que asigna un costo a cada plan de ejecución. Si s_i es un statement elemental y P , Q y P_i representan planes o subplanes, entonces:

$$C(s_i) = 1, \quad C(P; Q) = C(P) + C(Q), \quad C(P_1 \mid \cdots \mid P_k) = \max_{1 \leq i \leq k} C(P_i).$$

Expresión 1.1: Modelo de costos estructural simplificado.

En este modelo, los costos se acumulan en una secuencia, mientras que el máximo de una agrupación aplicativa representa su camino crítico. Por lo tanto, el costo compara la estructura de los planes. La Tabla 1.2 presenta los planes obtenidos para el programa del Código 1.1 en tres casos: ejecución secuencial, **ApplicativeDo** sobre el orden original y **ApplicativeDo** sobre el reordenamiento mostrado en la Figura 1.2.

Variante	Plan de ejecución	Costo
Secuencial	$s_1; s_2; s_3; s_4$	4
ApplicativeDo	$s_1; (s_2 \mid (s_3; s_4))$	3
Reordenamiento	$(s_1 \mid s_3); (s_2 \mid s_4)$	2

Tabla 1.2: Comparación de planes y costos estructurales para el ejemplo académico.

La comparación evidencia el problema abordado por esta memoria. La ejecución secuencial tiene costo 4 y **ApplicativeDo** reduce ese valor a 3 al introducir una agrupación aplicativa, pero conserva el orden relativo escrito por el programador. Por esta razón, no puede construir el plan de costo 2, que requiere intercambiar s_2 y s_3 . Esta restricción es necesaria para mónadas arbitrarias, cuyos efectos observables pueden depender del orden, pero puede ser demasiado conservadora bajo la hipótesis de conmutatividad considerada en esta memoria.

La solución desarrollada en esta memoria no reemplaza **ApplicativeDo**, sino que amplía su espacio de búsqueda en bloques tratados bajo una hipótesis explícita de conmutatividad. A partir de las dependencias del bloque, genera órdenes alternativos de statements que conservan sus precedencias y que, bajo dicha hipótesis, se consideran semánticamente equivalentes al orden original. Cada orden se evalúa mediante el algoritmo existente de **ApplicativeDo** y, finalmente, se selecciona el plan de ejecución de menor costo según el modelo estructural presentado.

Al aplicar esta solución al modelo académico del Código 1.1, se generaron cinco órdenes válidos. El orden $[1, 3, 2, 4]$ correspondió al primer candidato de costo mínimo y produjo el plan de costo 2 presentado en la Tabla 1.2. Para validar experimentalmente la preservación del resultado, se ejecutaron el programa original, la variante estándar de **ApplicativeDo**, la selección óptima y cada uno de los cinco candidatos forzados individualmente. Todas las variantes produjeron exactamente la misma distribución normalizada, lo que entrega evidencia de preservación semántica para este ejemplo. Los capítulos siguientes desarrollan la formulación del problema, la implementación de este proceso y su evaluación experimental.

1.3. Objetivos

El objetivo general de esta memoria es extender experimentalmente la implementación de `ApplicativeDo` en `GHC` para que pueda explorar reordenamientos de statements que preserven las dependencias locales dentro de bloques `do` marcados explícitamente bajo una hipótesis de conmutatividad. Con ello, se busca ampliar el espacio de planes de ejecución evaluados por el compilador y seleccionar, según el modelo de costos adoptado, un plan de menor costo cuando algún reordenamiento admisible lo permita.

Los objetivos específicos son los siguientes:

1. Analizar y describir el funcionamiento de `ApplicativeDo` dentro de `GHC`, en particular el procesamiento de bloques `do` y la generación de planes de ejecución.
2. Diseñar un modelo de dependencias locales entre statements que permita identificar reordenamientos que preserven las relaciones entre las variables definidas y sus usos.
3. Implementar una extensión experimental de `GHC` que reconozca bloques marcados bajo una hipótesis de conmutatividad, genere reordenamientos admisibles, evalúe cada alternativa mediante el algoritmo existente de `ApplicativeDo` y seleccione un plan de costo mínimo.
4. Evaluar experimentalmente la preservación de resultados y la reducción potencial del costo estructural frente a `ApplicativeDo` aplicado al orden original, mediante corpus sintéticos basados en `Maybe` y `Dist.T Rational`, junto con un control negativo basado en `IO`.

1.4. Contribuciones

Las principales contribuciones de esta memoria son:

- Un mecanismo para generar reordenamientos de statements que preservan las dependencias del programa y que, bajo una hipótesis explícita de conmutatividad, representan permutaciones semánticamente equivalentes.
- Una extensión experimental de `GHC` que integra este mecanismo con `ApplicativeDo`, amplía el espacio de planes de ejecución considerado por el compilador y selecciona una alternativa de costo mínimo según el modelo estructural adoptado.
- Una validación experimental reproducible que comprueba, en los casos conmutativos evaluados, que todas las permutaciones generadas producen el mismo resultado observable que el programa original y que la selección automática corresponde a un plan de costo estructural mínimo.

Capítulo 2

Marco Teórico

Este capítulo presenta los fundamentos conceptuales necesarios para comprender tanto el problema abordado en esta memoria como la solución desarrollada. En primer lugar, se introducen las características de **Haskell** necesarias para interpretar los ejemplos y las firmas de tipos utilizadas a lo largo del informe. Sobre esta base, se estudian sus principales abstracciones funcionales: funtores, aplicativos y mónadas. Estas clases forman una jerarquía que permite estructurar cómputos dentro de un contexto y se distinguen por el grado de dependencia que pueden expresar entre las distintas partes de un programa.

A continuación, se profundiza en la *do-notation*, explicando su relación con la composición monádica y cómo sus statements de enlace se traducen, en su forma convencional, mediante el operador *bind* ($>>=$). Esto es fundamental para entender por qué **ApplicativeDo** puede reemplazar parte de una expresión monádica por una expresión aplicativa [19, 9].

Sobre esta base, se introduce la noción de mónada conmutativa, cuya propiedad relevante permite intercambiar ciertos cómputos independientes sin modificar la semántica del programa [15, 13]. Posteriormente, se presentan las mónadas de probabilidad como un caso canónico de esta propiedad y como el principal dominio de aplicación de la memoria.

Finalmente, se estudia la extensión **ApplicativeDo**, describiendo la transformación que realiza sobre bloques **do**, la forma en que identifica dependencias entre statements y la generación de expresiones aplicativas que pueden exponer una mayor estructura paralelizable. Este recorrido permite establecer los conceptos necesarios para analizar posteriormente las limitaciones del mecanismo actual y la extensión experimental propuesta.

2.1. Fundamentos de Haskell

Haskell es un lenguaje de programación funcional puro, de propósito general y con tipado estático. En este lenguaje, las funciones desempeñan un papel central y son tratadas como valores de primera clase: pueden recibirse como argumentos, retornarse como resultados y almacenarse en estructuras de datos. Esta característica permite que los programas no solo operen con valores como números o listas, sino también con las funciones que transforman

dichos valores [16, capítulo 1 y sección 6.1.6].

La pureza implica que la evaluación de una función depende de sus argumentos y no produce efectos secundarios implícitos. Los efectos se representan de manera explícita mediante tipos y abstracciones que determinan cómo se componen los cálculos. Esta separación será relevante al introducir mónadas y analizar el orden de los efectos expresados mediante bloques `do` [16, capítulo 7] [26].

Además, las funciones de **Haskell** se encuentran curricadas. Esto significa que una función que conceptualmente recibe varios argumentos se representa como una sucesión de funciones que reciben un argumento a la vez. Por esta razón, las flechas de una firma de tipos se asocian hacia la derecha; por ejemplo, el tipo `a -> b -> c` se interpreta como `a -> (b -> c)`. Una función de este tipo recibe un valor de tipo `a` y retorna una nueva función de tipo `b -> c` [16, secciones 3.3 y 4.1.2].

Este comportamiento puede observarse en el operador binario `(+)`, cuya firma de tipos se muestra en el Código 2.1 [16, sección 6.4 y figura 6.2].

```
(+) :: Num a => a -> a -> a
```

Código 2.1: Firma de tipos del operador de suma `(+)`.

En esta firma, `a` es una variable de tipo y la restricción `Num a` establece que debe elegirse un tipo perteneciente a la clase `Num`. Si se aplica `(+)` únicamente al valor `5`, se fija su primer argumento y se obtiene una nueva función que todavía espera el argumento restante. Esta operación se denomina *aplicación parcial*. En este caso, la expresión `(+) 5` produce la función unaria `(5+)` de tipo `Num a => a -> a` [16, secciones 3.5, 4.1.3 y 6.4].

Los tipos también pueden estar parametrizados. Por ejemplo, **Maybe** es un constructor de tipos que recibe un tipo `a` y produce el tipo `Maybe a`. Sus constructores de datos representan dos posibilidades: **Just** contiene un valor de tipo `a` y **Nothing** representa su ausencia. Esta distinción entre un constructor de tipos y los tipos concretos que produce permite definir comportamientos comunes para toda la familia **Maybe** [16, secciones 4.1.2 y 6.1.8; capítulo 21].

Una clase de tipos especifica un conjunto de operaciones que distintos tipos o constructores de tipos pueden implementar. Cada implementación concreta se declara mediante una instancia [16, secciones 4.1 y 4.3]. La restricción `Num a` del ejemplo anterior indica precisamente que debe existir una instancia de `Num` para el tipo elegido. De manera análoga, **Functor**, **Applicative** y **Monad** son clases de tipos con operaciones e instancias propias. **Maybe** posee instancias para las tres y, por ello, se utilizará como ejemplo común para mostrar cómo un mismo contexto puede transformar valores, combinar cálculos y expresar dependencias entre sus resultados [27, 25].

2.2. Abstracciones Funcionales

Las funciones de primera clase, la currificación, la aplicación parcial y las clases de tipos proporcionan la base para las abstracciones estudiadas en esta sección. **Functor**, **Applicative** y **Monad** forman una jerarquía que se distingue por el grado de dependencia que permiten expresar entre las partes de un programa. Un functor permite transformar el resultado de un cómputo; un aplicativo permite combinar cómputos cuya estructura está determinada de antemano; y una mónada permite que un cómputo posterior dependa del valor producido por uno anterior [26, 21, 27].

2.2.1. Functores

En Haskell, **Functor** es una clase de tipos cuyas instancias se definen para constructores de tipos, como las listas `[]` o `Maybe`. Su operación principal es `fmap`, que permite aplicar una función a los valores asociados a una estructura y obtener un resultado que conserva el mismo constructor de tipos. La definición simplificada de la clase se muestra en el Código 2.2:

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

Código 2.2: Definición simplificada de la clase **Functor**.

En esta firma, `f` representa un constructor de tipos, mientras que `a` y `b` representan los tipos de los valores antes y después de aplicar la función. Así, `fmap` recibe una función de tipo `a -> b` y transforma un valor de tipo `f a` en uno de tipo `f b`, manteniendo el contexto representado por `f`.

En instancias concretas como las listas y `Maybe`, esta operación puede entenderse intuitivamente mediante la metáfora de un contenedor: `fmap` transforma los valores contenidos sin modificar la estructura que los organiza. Bajo esta interpretación pedagógica, puede imaginarse que la operación accede a los valores, aplica una función sobre ellos y vuelve a representarlos dentro del mismo contexto. Sin embargo, esta metáfora no constituye la definición general de un functor.

Además, toda instancia válida de **Functor** debe respetar las leyes de identidad y composición. Estas leyes garantizan que la aplicación de `fmap` preserve de manera coherente la estructura asociada al constructor de tipos [27].

Los ejemplos clásicos de funtores incluyen las listas y los árboles, pues `fmap` transforma sus elementos de manera uniforme mientras mantiene su estructura subyacente.

Como se mencionó anteriormente, otro ejemplo relevante es `Maybe`. El tipo `Maybe a` representa valores opcionales: un valor disponible se expresa mediante el constructor de datos `Just`, mientras que la ausencia de un valor se representa mediante `Nothing`. Al utilizar `fmap` sobre este tipo, la función recibida se aplica al valor contenido en `Just`, mientras que `Nothing`

permanece sin cambios.

En la implementación de **base** incluida con GHC, este comportamiento se define mediante la instancia de **Functor** para el constructor de tipos **Maybe** mostrada en el Código 2.3 [25]:

```
instance Functor Maybe where
  fmap _ Nothing = Nothing
  fmap f (Just a) = Just (f a)
```

Código 2.3: Instancia de **Functor** para **Maybe**.

La primera ecuación establece que, cuando el valor recibido es **Nothing**, no existe ningún elemento sobre el cual aplicar la función y, por tanto, el resultado permanece como **Nothing**. El guion bajo indica que la función recibida no necesita utilizarse en este caso. En cambio, cuando el valor tiene la forma **Just a**, la segunda ecuación aplica la función **f** al elemento **a** y vuelve a encapsular el resultado mediante el constructor **Just**. Ambos comportamientos se ilustran en el Código 2.4:

```
> fmap (+1) (Just 2)
Just 3
> fmap (+1) Nothing
Nothing
```

Código 2.4: Aplicación de **fmap** sobre valores de tipo **Maybe**.

2.2.2. Aplicativos

En Haskell, **Applicative** es una clase de tipos definida para constructores de tipos que ya poseen una instancia de **Functor**. Sus operaciones principales son **pure** y **(<*>)**: la primera introduce un valor en un contexto, mientras que la segunda aplica una función contextualizada a un valor dentro de ese mismo contexto. De esta forma, la clase caracteriza un estilo aplicativo de programación, menos expresivo que el monádico y, por tanto, aplicable a una mayor variedad de estructuras [21]. Para ilustrar este comportamiento, se utilizará nuevamente el constructor de tipos **Maybe** y el operador binario **(+)**.

Un constructor de tipos que posee una instancia de **Applicative** se denomina aplicativo. Considerando **Maybe**, para sumar **Just 2** y **Just 3**, la función **(+)** se introduce primero en el contexto mediante **pure**. La primera aplicación de **(<*>)** produce la función unaria contextualizada **Just ((+) 2)**, de tipo **Maybe (Int -> Int)**. La segunda aplicación combina esta función con **Just 3** y produce el resultado **Just 5**.

Cada instancia de **Applicative** define cómo se comportan estas operaciones para un constructor de tipos concreto. En la instancia de **Maybe**, si alguno de los cómputos asociados a los sumandos produce **Nothing**, esta ausencia se propaga al resultado. De esta forma, el comportamiento de los valores opcionales queda definido una sola vez en la instancia, evitando

repetir ajustes de patrones cada vez que se combinan operaciones que pueden fallar.

Formalmente, la clase `Applicative` extiende `Functor` y declara las operaciones `pure` y `(<*>)` mediante la definición simplificada del Código 2.5:

```
class Functor f => Applicative f where
  pure :: a -> f a
  <*>  :: f (a-> b) -> f a -> f b
```

Código 2.5: Definición simplificada de la clase `Applicative`.

En particular, la instancia `Applicative Maybe` define los operadores `pure` y `<*>` de acuerdo con el comportamiento de los valores opcionales: permite introducir valores en el contexto y combinar operaciones que pueden fallar. Una definición de esta instancia se presenta en el Código 2.6:

```
instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  _ <*> Nothing = Nothing
  (Just f) <*> (Just x) = Just (f x)
```

Código 2.6: Instancia de `Applicative` para `Maybe`.

Nótese que, en caso de operar con algún fallo de cómputo o ausencia de algún valor, cuya abstracción equivale al constructor `Nothing`, las siguientes aplicaciones retornarán a su vez un elemento `Nothing`, propagando así el comportamiento definido por la instancia `Applicative Maybe`. En caso contrario, si ambos operandos se encuentran definidos, es decir, son elementos encapsulados en el constructor `Just`, se procede con la aplicación del primero sobre el segundo, para finalmente empaquetar el elemento resultante dentro del mismo constructor `Just`.

Volviendo al ejemplo de aplicación de la suma (+) sobre elementos de tipo `Maybe Int`, se requiere que esta se encuentre encapsulada dentro del constructor `Just`. El operador `<*>` espera que su operando izquierdo sea una función dentro del contexto. Por ello, `pure (+)` introduce la función de suma en `Maybe`, produciendo el valor contextualizado requerido. La expresión aplicativa concreta se muestra en el Código 2.7:

```
pure (+) <*> Just 2 <*> Just 3
```

Código 2.7: Suma aplicativa de dos valores de tipo `Maybe Int`.

Como `Functor` es una superclase de `Applicative`, todo constructor de tipos con una instancia de `Applicative` también dispone de `fmap`. El operador `(<$>)` corresponde a su forma infija, por lo que se cumple la igualdad `f <$> m = fmap f m`. Además, la coherencia entre ambas clases establece que `fmap f m = pure f <*> m`. En consecuencia, ambas expre-

siones pueden relacionarse mediante `f <$> m = fmap f m = pure f <*> m`. De este modo, el ejemplo anterior puede escribirse con la sintaxis más directa del Código 2.8:

```
(+) <$> Just 2 <*> Just 3
```

Código 2.8: Forma infija de la suma aplicativa sobre `Maybe`.

Finalmente, nótese que en una expresión aplicativa, la estructura del cómputo queda fijada de antemano: cada argumento se evalúa en su propio contexto y luego se combinan sus resultados mediante la aplicación de una función pura. Esta característica hace explícita la ausencia de dependencias de datos entre los subcómputos asociados a los argumentos, pues ninguno puede usar el resultado del otro para determinar su propia forma. Esto no implica que sus efectos sean conmutativos, aunque puede habilitar una evaluación más paralelizable cuando la implementación lo permite [21].

Esta misma restricción implica que, si se quisiera que el segundo argumento dependa del valor producido por el primero, el estilo aplicativo resulta insuficiente, ya que la forma del cómputo no puede adaptarse dinámicamente a resultados intermedios. Por otro lado, la abstracción monádica proporciona el operador `bind (>>=)`, que permite construir el segundo cómputo a partir del resultado del primero. Esta capacidad de expresar dependencias entre resultados intermedios motiva la introducción de la clase `Monad` en `Haskell` [26].

2.2.3. Mónadas

En `Haskell`, `Monad` es una clase de tipos cuyas instancias definen cómo secuenciar cómputos dentro de un mismo contexto cuando un cómputo posterior puede depender del valor producido por uno anterior [26]. Como `Applicative` es una superclase de `Monad`, todo constructor de tipos que posea una instancia de `Monad` también debe poseer una instancia de `Applicative`. El Código 2.9 muestra una definición simplificada con las operaciones relevantes:

```
class Applicative m => Monad m where
  (>>=) :: m a -> (a -> m b) -> m b
  return :: a -> m a
```

Código 2.9: Definición simplificada de la clase `Monad`.

Para interpretar la firma de `(>>=)`, consideraremos un cómputo `ma :: m a` y una función `k :: a -> m b`. La expresión `ma >>= k` compone el primer cómputo con la función `k` de acuerdo con el comportamiento definido por la instancia, por ende entrega su resultado de tipo `a` a la función `k` y obtiene un nuevo cómputo de tipo `m b`. De esta manera, la función `k` puede utilizar el resultado anterior para determinar la forma del cómputo siguiente.

La operación `return` introduce un valor en el contexto monádico y, en la jerarquía actual de clases de tipos, coincide con `pure`. Su nombre no representa una transferencia de control como en lenguajes imperativos, sino la construcción de un valor de tipo `m a` a partir de uno

de tipo `a`.

Al encadenar varias aplicaciones de (`>>=`), las funciones entregadas al operador pueden escribirse como funciones anónimas λ . Estas permiten asignar nombres a los resultados intermedios y utilizarlos en los cálculos posteriores. Para ilustrar este comportamiento, se retomará el ejemplo de la suma con `Maybe`.

En este caso, la instancia `Monad Maybe` define el comportamiento del operador `>>=` para los valores opcionales. Las operaciones relevantes se presentan en el Código 2.10:

```
instance Monad Maybe where
  return x = Just x
  Nothing >>= _ = Nothing
  (Just x) >>= f = f x
```

Código 2.10: Instancia de `Monad` para `Maybe`.

Cuando el primer cálculo produce `Nothing`, la ausencia de valor se propaga como resultado. En cambio, cuando produce un valor de la forma `Just x`, el elemento `x` se entrega como argumento a la función ubicada a la derecha de `>>=`. En este caso, la función que se ejecuta debe respetar la firma de tipos y retornar un nuevo cálculo de tipo `m b`. El Código 2.11 presenta la aplicación de la función `(+)` mediante notación monádica:

```
Just 3 >>= (\x -> Just 2 >>= (\y -> return ((+) x y)))
```

Código 2.11: Suma monádica de dos valores de tipo `Maybe`.

Por tanto, al ejecutar el operador `>>=`, se desempaqueta el número 3 y se aplica la función anónima de la derecha sobre este valor, reemplazando todas las apariciones del identificador `x` por 3. A continuación, se realiza el mismo proceso con el cálculo `Just 2`, resultando finalmente en la expresión `((+) 3 2)` que será empaquetada gracias a la instrucción `return` retornando el valor contextualizado `Just 5`.

Cabe recalcar que ahora el identificador `x` queda dentro del alcance de la función que construye el segundo cálculo. Por ello, la expresión asociada a `Just 2` puede reemplazarse por otra que utilice `x`. El Código 2.12 muestra un programa que utiliza esta nueva expresividad de las mónadas:

```
Just 3 >>= (\x -> Just ((* 2 x) >>= (\y -> return ((+) x y)))
```

Código 2.12: Cálculo monádico con dependencia entre resultados intermedios.

Del programa anterior, el primer cálculo desempaqueta `Just 3` y reemplaza este elemento en las apariciones del identificador `x`. Luego, el cálculo posterior `Just ((* 2 x)` es capaz de considerar el valor numérico de `x` retornando `Just 6`, luego se desempaqueta su resultado y este reemplaza las apariciones del identificador `y`. Quedando como cálculo

final la operación `((+) 3 6)` cuyo resultado termina siendo empaquetado por la instrucción `return` como `Just 9`. Este flujo de ejecución permite una mayor expresividad enriqueciendo el programa. De esta manera la notación monádica termina siendo algo característico y central en `Haskell`.

Aunque esta composición permita expresar dependencias de forma correcta y versátil, la acumulación de funciones anónimas dificulta su lectura. La *do-notation*, presentada a continuación, proporciona una sintaxis más clara para escribir la misma secuencia monádica.

2.3. Do-Notation

La *do-notation* es una sintaxis de `Haskell` para escribir cálculos monádicos de forma secuencial. Un bloque `do` contiene statements que pueden vincular resultados intermedios y utilizarlos en los statements posteriores. Esta notación no define una forma nueva de composición, sino que proporciona una escritura más directa para expresiones construidas mediante las operaciones monádicas `(>>=)`, `(>>)` y `fail`, además de contemplar una regla propia para declaraciones `let` [16, sección 3.14].

Para observar la motivación de esta sintaxis, se retomará la suma de dos valores de tipo `Maybe` presentada mediante el operador *bind* en el Código 2.11. Aunque la expresión es compacta, la acumulación de operadores, funciones anónimas y paréntesis dificulta distinguir su estructura a simple vista.

El Código 2.13 distribuye la misma expresión en varias líneas, sin cambiar el orden ni la agrupación de sus operaciones, para mostrar con mayor claridad el flujo de definición y composición de los cálculos:

```
Just 3 >>= (\x ->
  Just 2 >>= (\y ->
    return ((+) x y)))
```

Código 2.13: Forma multilínea de la suma monádica sobre `Maybe`.

La indentación permite reconocer que la composición con `Just 2` se encuentra dentro de la función que introduce `x`, mientras que `return` cierra la composición utilizando los identificadores `x` e `y`. Este cambio es únicamente visual: la expresión conserva la semántica del Código 2.11 y produce `Just 5`.

La *do-notation* permite expresar este mismo flujo sin escribir de forma explícita las funciones anónimas ni las aplicaciones sucesivas de `(>>=)`. En un statement de la forma `p <- e`, la expresión `e` representa el cómputo y el patrón `p` vincula su resultado para que pueda utilizarse en los statements posteriores. El Código 2.14 presenta la escritura equivalente:

```
do
  x <- Just 3
  y <- Just 2
  return ((+) x y)
```

Código 2.14: Suma monádica sobre `Maybe` escrita mediante *do-notation*.

En el Código 2.14, el primer statement corresponde al cómputo entregado al (`>>=`) exterior y vincula su resultado con `x`; el segundo corresponde al (`>>=`) interior y vincula su resultado con `y`; el último construye el resultado del bloque. Al aplicar las reglas convencionales de traducción de la *do-notation*, este bloque recupera la composición anidada del Código 2.13 y, en última instancia, la expresión compacta del Código 2.11 [16, sección 3.14].

Aunque el bloque se escribe como una secuencia, en este ejemplo la expresión `Just 2` no utiliza el valor de `x`; por ello, entre los dos primeros statements no existe una dependencia de datos local. Ambos resultados sí son utilizados por el statement final. Esta distinción entre orden sintáctico y dependencias de datos será relevante más adelante: la notación hace más legible la composición, pero no autoriza por sí sola a intercambiar cálculos, ya que en una mónada general el orden de sus efectos puede ser observable. Las condiciones que permiten transformar o reordenar partes de esta secuencia se estudiarán al introducir las mónadas conmutativas y `ApplicativeDo`.

2.4. Mónadas Conmutativas

Kock formaliza la conmutatividad de una mónada mediante dos mapas de Fubini que combinan un par de cálculos en órdenes opuestos. Cuando ambos mapas coinciden, el orden elegido para efectuar esta combinación no modifica su significado [15, secciones 5 y 9]. En términos operacionales, esto significa que, dados dos cálculos monádicos independientes, ejecutar el primero y luego el segundo es semánticamente equivalente a ejecutarlos en el orden contrario, siempre que se conserven las asociaciones entre sus resultados.

La mónada `Maybe` permite ilustrar esta propiedad mediante el mismo ejemplo de suma utilizado en las secciones anteriores. El Código 2.15 presenta el bloque original y una variante en la que se intercambian los dos primeros statements:

<pre>sumaOriginal = do x <- Just 3 y <- Just 2 return ((+) x y)</pre>	$\Leftarrow$	<pre>sumaReordenada = do y <- Just 2 x <- Just 3 return ((+) x y)</pre>
---	--------------	---

Código 2.15: Suma monádica sobre `Maybe` en el orden original y en el reordenado.

En `sumaOriginal`, el primer statement vincula el valor 3 con `x` y el segundo vincula el

valor 2 con y . En `sumaReordenada` se invierte el orden de ejecución, pero se mantienen las mismas asociaciones: x continúa vinculado con 3 y y con 2. Como la expresión final tampoco cambia, ambos bloques producen `Just 5`.

Esta equivalencia no se debe a que la suma sea una operación conmutativa. El intercambio de statements no modifica el orden de los argumentos de `((+) x y)`, sino solamente el orden en que se obtienen los valores asociados a x e y . La propiedad relevante pertenece a la mónada: bajo la semántica total de valores opcionales, si ambos cómputos producen un valor de la forma `Just`, estos valores se conservan en cualquiera de los dos órdenes; si alguno produce `Nothing`, ambos órdenes producen `Nothing`. Esta caracterización no considera valores divergentes, como `undefined`, ni efectos externos.

Por lo tanto, conviene distinguir independencia estructural de conmutatividad. La expresión `Just 2` no utiliza x , de modo que intercambiar los dos primeros statements conserva su alcance y sus dependencias de datos. Sin embargo, esta ausencia de dependencia no bastaría para una mónada arbitraria: dos acciones de tipo `IO` podrían no utilizar sus resultados mutuamente y aun así imprimir mensajes en órdenes diferentes. La independencia permite formular ambos órdenes, mientras que la conmutatividad garantiza que el cambio de orden no altera los efectos observables [19, p. 94].

La noción también aparece de manera natural en el estudio de distribuciones. Kock presenta las mónadas conmutativas como una teoría general de distribuciones [15], mientras que Jacobs identifica específicamente la mónada de distribuciones discretas finitas como una mónada conmutativa [13, sección 3.1]. Esta relación proporciona la base para estudiar a continuación las mónadas de probabilidad, sin implicar que toda construcción monádica probabilística sea necesariamente conmutativa.

2.5. Mónadas de Probabilidad

Una mónada de probabilidad permite representar un cómputo como una distribución sobre sus posibles resultados. En vez de producir un único valor, cada operación puede introducir alternativas y asignar una probabilidad a cada una de ellas; la composición monádica combina posteriormente estas alternativas y propaga sus probabilidades. Para distribuciones discretas finitas, este enfoque permite enumerar de forma exacta el soporte inducido por un programa y ha sido utilizado para expresar modelos probabilísticos de manera composicional en lenguajes funcionales [3, 24].

El ejemplo de la moneda y el dado presentado en la Introducción constituye el caso más simple de esta interpretación. Sus dos elecciones uniformes generan los doce pares posibles y cada par recibe probabilidad $1/12$. Más que repetir ese ejemplo, esta sección se concentra en la representación concreta de las distribuciones y en operaciones que permiten construir modelos con elecciones uniformes, pesos no uniformes y dependencias entre variables.

La implementación utilizada en esta memoria pertenece al paquete externo `probability`, publicado en Hackage. Por lo tanto, no forma parte de los módulos estándar de `Haskell` ni del paquete `base`. Su módulo público principal es `Numeric.Probability.Distribution`, impor-

tado habitualmente con el calificador `Dist`; este módulo proporciona el tipo `T`, sus instancias de `Functor`, `Applicative` y `Monad`, y constructores de distribuciones como `certainly`, `choose`, `uniform` y `fromFreqs` [4].

La definición esencial del tipo `T` se muestra en el Código 2.16. Una distribución se almacena internamente como una lista de pares: el primer componente de cada par es un resultado posible y el segundo es su probabilidad asociada [4].

```
newtype T prob a = Cons
  { decons :: [(a, prob)] }
```

Código 2.16: Representación interna de una distribución dentro del paquete `probability`.

El parámetro `a` corresponde al tipo de los resultados y `prob` al tipo usado para sus probabilidades. En particular, `Dist.T Rational a` representa resultados de tipo `a` con pesos racionales. El uso de `Rational` permite conservar exactamente fracciones como $1/3$ o $3/10$, sin introducir los errores de redondeo propios de una representación de punto flotante. Por ejemplo, el constructor `Dist.Cons` puede almacenar los resultados `x` e `y` en una lista con los pesos $1/4$ y $3/4$, respectivamente.

Esta lista registra ramas de cómputo y no necesariamente constituye una representación canónica de la distribución. Un mismo resultado puede aparecer varias veces si fue alcanzado mediante caminos distintos. Además, el orden de los pares refleja el orden en que esas ramas fueron generadas. La semántica probabilística se obtiene al considerar, para cada resultado, la suma de los pesos de todas las apariciones que lo producen.

Las operaciones del módulo construyen estas listas de manera controlada. Una elección determinista puede escribirse como `Dist.certainly x` y produce el único par $(x, 1)$. La expresión `Dist.choose p x` y produce dos ramas: `x` con probabilidad p e `y` con probabilidad $1 - p$. Por su parte, `Dist.uniform` asigna peso $1/n$ a cada elemento de una lista de largo n , mientras que `Dist.fromFreqs` recibe pares de valores y frecuencias, y divide cada frecuencia por la suma total para obtener probabilidades. Así, las frecuencias 5, 3 y 2 se transforman respectivamente en las probabilidades $1/2$, $3/10$ y $1/5$ [4].

La instancia de `Monad` determina cómo se propagan estos pesos. Si una distribución contiene una rama (x, p) y el cómputo posterior aplicado a `x` contiene una rama (y, q) , el operador `(>>=)` incorpora el resultado `y` con peso pq . En consecuencia, la composición enumera las combinaciones posibles y multiplica las probabilidades a lo largo de cada camino. Cuando varios caminos terminan en el mismo valor, sus pesos permanecen inicialmente en pares separados.

El Código 2.17 combina las tres formas de construcción anteriores. Primero define una distribución ponderada para el clima. Luego usa `Dist.choose` para seleccionar una actividad según el clima obtenido y, finalmente, elige uniformemente su duración. La distribución retornada omite el clima, que actúa como una variable intermedia del modelo.

```

import qualified Numeric.Probability.Distribution as Dist

data Weather = Sunny | Cloudy | Rainy
    deriving (Eq, Ord, Show)

data Activity = Outdoor | Indoor
    deriving (Eq, Ord, Show)

weekendPlan :: Dist.T Rational (Activity, Int)
weekendPlan = do
    weather <- Dist.fromFreqs
        [(Sunny, 5), (Cloudy, 3), (Rainy, 2)]
    activity <- case weather of
        Sunny   -> Dist.choose (4 / 5) Outdoor Indoor
        Cloudy  -> Dist.choose (1 / 2) Outdoor Indoor
        Rainy   -> Dist.choose (1 / 10) Outdoor Indoor
    hours <- Dist.uniform [1, 2, 3]
    return (activity, hours)

normalizedWeekendPlan = Dist.norm weekendPlan

```

Código 2.17: Modelo probabilístico con frecuencias, elecciones condicionadas y una distribución uniforme.

Antes de normalizar, este programa genera dieciocho ramas: tres posibilidades para el clima, dos para la actividad y tres para la duración. Sin embargo, como el clima no forma parte del resultado, solo existen seis pares distintos de actividad y duración. Por ejemplo, el resultado (`Outdoor`, 1) puede alcanzarse desde cualquiera de los tres climas. La suma exacta de sus ramas es

$$\frac{1}{3} \left(\frac{1}{2} \frac{4}{5} + \frac{3}{10} \frac{1}{2} + \frac{1}{5} \frac{1}{10} \right) = \frac{19}{100}.$$

Expresión 2.1: Cálculo de la probabilidad del resultado (`Outdoor`, 1).

La función `Dist.norm` agrupa los resultados iguales y suma sus probabilidades, produciendo en este caso una lista canónica de seis resultados. En este módulo, esta operación de normalización corresponde específicamente al agrupamiento de resultados repetidos; no vuelve a dividir pesos arbitrarios para hacer que sumen uno. Esa segunda operación sí ocurre al construir una distribución mediante `Dist.fromFreqs` [4].

La distinción entre distribución y lista interna es relevante para la conmutatividad. Al intercambiar dos cálculos probabilísticos independientes, el producto de sus probabilidades se conserva, aunque el recorrido de las ramas puede producir los pares en un orden distinto. Por ello, la equivalencia considerada en esta memoria se establece sobre las distribuciones

normalizadas y no sobre el orden literal expuesto por `decons`. Bajo esta interpretación, la mónada de distribuciones discretas finitas constituye el caso conmutativo descrito en la sección anterior [13, sección 3.1].

El mismo tipo `Dist.T Rational` se utiliza en el modelo académico del Código 1.1. Allí, `Dist.uniform` representa la preparación y la dificultad, mientras que `Dist.choose` expresa las probabilidades condicionadas de aprobar. Tanto ese ejemplo como el corpus probabilístico aplican `Dist.norm` antes de observar el resultado. De esta forma, la validación compara probabilidades racionales exactas y evita confundir diferencias en el orden interno o en la multiplicidad de las ramas con diferencias semánticas entre distribuciones.

2.6. ApplicativeDo

`ApplicativeDo` es una extensión de `GHC` basada en el trabajo de Marlow, Peyton Jones, Kmett y Mokhov [19]. Su objetivo es permitir que programas escritos con *do-notation* usen combinadores aplicativos cuando las dependencias lo permiten. De esta forma, el programador conserva la sintaxis monádica habitual, mientras el compilador puede generar una estructura de ejecución más paralelizable.

La transformación propuesta recibe un bloque `do`, analiza las dependencias entre sus `statements` y produce una expresión equivalente que utiliza operaciones aplicativos donde existe independencia, sin dejar de utilizar operaciones monádicas donde un cómputo depende del resultado de otro. Para ello, el algoritmo se organiza conceptualmente como un *pipeline* de dos etapas: *rearrange* construye un plan que combina secuencias y agrupaciones aplicativos, mientras que *desugaring* traduce ese plan a combinadores explícitos [19, pp. 95–96].

Esta separación también forma parte del diseño de la implementación. La primera etapa conserva la estructura de los `statements` y registra cómo deben agruparse; la segunda consume esa información para producir las operaciones que implementan el plan. A continuación se profundizará en el diseño conceptual de las etapas ya mencionadas *rearrange* y *desugaring*, aplicando luego el algoritmo de `ApplicativeDo` al programa probabilístico del Código 1.1.

2.6.1. Etapa de Rearrange

Para presentar el algoritmo, considérese una expresión de *do-notation* cuyos `statements` de enlace tienen la forma `p_i <- e_i`. Cada `statement` se denota mediante s_i , donde i indica su posición dentro del bloque. El algoritmo representa entonces la expresión mediante la forma abstracta `do 1 e`, en la que $l = \{s_1; \dots; s_n\}$ es la secuencia de `statements` y e es la expresión final que produce el resultado. Esta última corresponde al argumento de un `return` o `pure` terminal y no participa en los cálculos de *rearrange*.

La Figura 2.1 presenta las funciones que componen esta etapa. La función principal, `rearrange`, recibe la secuencia l y construye otra secuencia l' que puede contener la forma paralela $(l_1 \mid \dots \mid l_k)$. Esta forma no pertenece a la sintaxis escrita por el programador, sino a

una sintaxis abstracta intermedia que expresa que las secuencias $l_1, \dots, l_k$ son independientes y pueden combinarse mediante operaciones aplicativas.

$$\begin{aligned}
\text{rearrange } \{s_1; \dots; s_n\} &= \{s_1\}, && \text{if } n = 1 \\
&= \text{split } g_1, && \text{if } k = 1 \\
&= \{(\text{split } g_1 \mid \dots \mid \text{split } g_k)\}, && \text{otherwise} \\
\text{where} \\
g_1 \dots g_k &= \text{segments } \{s_1; \dots; s_n\} \\
\text{segments } \{s_1; \dots; s_n\} &= \{s_1; \dots; s_{i_1}\} \dots \{s_{(i_k)+1}; \dots; s_n\} \\
\text{where} \\
i_1 \dots i_k &= \{ i \in 1 \dots n \\
&\quad \mid \text{bv}\{s_1; \dots; s_i\} \cap \text{fv}\{s_{i+1}; \dots; s_n\} = \emptyset \} \\
\text{split}\{s_1; \dots; s_n\} &= \{s_1\}, && \text{if } n = 1 \\
&= \text{splitat } i_{\text{opt}}, && \text{otherwise} \\
\text{where} \\
\text{splitat } i &= \text{rearrange } \{s_1; \dots; s_i\} \text{ ++ rearrange } \{s_{i+1}; \dots; s_n\} \\
i_{\text{opt}} &\in 1 \dots n \text{ such that} \\
\forall j. 1 \leq j < n. \text{cost}_s(\text{splitat } j) &\geq \text{cost}_s(\text{splitat } i_{\text{opt}}) \\
\text{cost}_s \{s_1; \dots; s_n\} &= \Sigma \{ \text{cost}_a s_i \mid 1 \leq i \leq n \} \\
\text{cost}_a (p \leftarrow e) &= 1 \\
\text{cost}_a (l_1 \mid \dots \mid l_n) &= \max \{ \text{cost}_s l_i \mid 1 \leq i \leq n \} \\
\text{fv } \{s_1; \dots; s_n\} &= \text{the free variables of } s_1 \dots s_n \\
\text{bv } \{s_1; \dots; s_n\} &= \text{the variables bound by } s_1 \dots s_n
\end{aligned}$$

Figura 2.1: Etapa de *rearrange* propuesta por Marlow et al. [19, p. 95].

Si la entrada contiene un único statement, **rearrange** lo retorna sin modificaciones. Para una secuencia de mayor tamaño, primero invoca a **segments**, que examina las $n - 1$ separaciones posibles entre statements consecutivos. Una separación ubicada después de s_i divide la entrada en el prefijo $\{s_1; \dots; s_i\}$ y el sufijo $\{s_{i+1}; \dots; s_n\}$. El corte es válido cuando se cumple:

$$\text{bv}(\{s_1; \dots; s_i\}) \cap \text{fv}(\{s_{i+1}; \dots; s_n\}) = \emptyset,$$

Expresión 2.2: Condición de independencia entre el prefijo y el sufijo de una secuencia.

En la notación **bv** representa las variables vinculadas por una secuencia y **fv** sus variables libres. Si la intersección es vacía, ninguna variable producida por el prefijo es requerida por el sufijo y, por tanto, no existe una dependencia que atravesase esa separación. Al considerar todos los cortes válidos, **segments** divide la entrada en los grupos $g_1, \dots, g_k$ que ya pueden formar las ramas de una agrupación aplicativa.

La condición anterior expresa una relación *def-use*. Esta relación establece que el uso de una variable debe encontrarse después de su definición y dentro del alcance introducido por ella. Si una variable definida en $\{s_1; \dots; s_i\}$ aparece libre en $\{s_{i+1}; \dots; s_n\}$, ambos fragmentos no pueden tratarse como ramas independientes: el segundo necesita un resultado producido

por el primero. En cambio, una intersección vacía indica que el corte no separa ninguna definición de sus usos. Como la secuencia original ya está bien formada, una variable definida en el sufijo tampoco puede ser utilizada por un statement anterior del prefijo.

Cuando **segments** obtiene más de un grupo, **rearrange** construye la forma (**split** $g_1 \mid \dots \mid \text{split } g_k$). Cada grupo se procesa mediante **split** para descubrir posibles agrupaciones anidadas. Si, por el contrario, toda la entrada constituye un único segmento, no existe un corte exterior que permita combinar dos partes aplicativamente y **rearrange** delega directamente la secuencia completa a **split**.

La función **split** considera nuevamente las separaciones $i \in \{1, \dots, n - 1\}$, pero con un propósito distinto. Para cada i , aplica **rearrange** recursivamente al prefijo y al sufijo, y luego compone ambos resultados de manera secuencial. De todas estas alternativas selecciona una de costo mínimo según el modelo estructural presentado en la Expresión 1.1. De esta manera, **segments** prioriza la independencia exterior que puede expresarse directamente mediante una agrupación aplicativa, mientras **split** determina dónde conservar una composición monádica cuando una dependencia impide realizar esa agrupación.

El resultado de la etapa *rearrange* es la expresión abstracta $\text{do } l' \text{ e}$. Aunque l' puede tener una estructura de árbol con secuencias y agrupaciones aplicativas, el algoritmo no permuta statements: si se eliminan las formas paralelas y se concatenan sus ramas de izquierda a derecha, se recupera la secuencia original $\{s_1; \dots; s_n\}$ [19, p. 95].

2.6.2. Etapa de Desugaring

La segunda etapa recibe el plan l' construido por *rearrange* junto con la expresión final e . Su función principal puede representarse mediante la firma:

$$\text{desugar} :: \text{Stmts} \rightarrow \text{Expr} \rightarrow \text{Expr}$$

Expresión 2.3: Firma abstracta de la función **desugar**.

El segundo argumento actúa como la continuación del plan: es la expresión que debe evaluarse una vez ejecutados los efectos de la secuencia y vinculadas las variables que utiliza.

La sintaxis intermedia permite que un statement sea un enlace ordinario o una agrupación ($l_1 \mid \dots \mid l_n$). En esta última, cada l_i es a su vez una secuencia de statements. Las variables definidas dentro de una rama no quedan disponibles para las demás ramas, pero aquellas requeridas por la continuación deben ser entregadas como resultados de esa rama. Esta restricción es la que permite reemplazar la agrupación por una expresión construida con $\langle \$ \rangle$ y $\langle * \rangle$.

La Figura 2.2 define el proceso mediante cinco casos recursivos. En conjunto, estos casos eliminan la sintaxis intermedia y producen una expresión formada por operaciones de Functor, Applicative y Monad.

$$\begin{aligned}
& \text{desugar} :: \text{Stmts} \rightarrow \text{Expr} \rightarrow \text{Expr} \\
& \text{desugar } \{ \} e = \text{pure } e \tag{0} \\
& \text{desugar } \{ p \leftarrow e \} e' \tag{1} \\
& \quad \begin{array}{l} | p == e' = e \\ | \text{otherwise} = (\lambda p \rightarrow e') \langle \$ \rangle e \end{array} \\
& \text{desugar } \{ p \leftarrow e; l \} e' = e \gg= \lambda p \rightarrow \text{desugar } l e' \tag{2} \\
& \text{desugar } \{ (l_1 \mid \dots \mid l_n) \} e \tag{3} \\
& \quad = (\lambda p_1 \dots p_n \rightarrow e) \langle \$ \rangle e_1 \langle * \rangle \dots \langle * \rangle e_n \\
& \quad \textbf{where } (p_i, e_i) = \text{desugar}_{\text{arg}} l_i \text{fv}(e) \\
& \text{desugar } \{ (l_1 \mid \dots \mid l_n); s \} e \tag{4} \\
& \quad = \text{join } ((\lambda p_1 \dots p_n \rightarrow e') \langle \$ \rangle e_1 \langle * \rangle \dots \langle * \rangle e_n) \\
& \quad \textbf{where} \\
& \quad \quad e' = \text{desugar } s e \\
& \quad \quad (p_i, e_i) = \text{desugar}_{\text{arg}} l_i \text{fv}(e') \\
& \text{desugar}_{\text{arg}} :: \text{Stmts} \rightarrow \text{Set Var} \rightarrow (\text{Pat}, \text{Expr}) \\
& \text{desugar}_{\text{arg}} \{ p \leftarrow e \} vs = (p, e) \\
& \text{desugar}_{\text{arg}} l vs = ((v_1, \dots, v_k), \text{desugar } l (v_1, \dots, v_k)) \\
& \quad \textbf{where} \\
& \quad \quad v_1 \dots v_k = \text{bv}(l) \cap vs
\end{aligned}$$

Figura 2.2: Etapa de *desugaring* propuesta por Marlow et al. [19, p. 96].

- Caso (0).** Si la secuencia de statements está vacía, no queda ningún efecto por ejecutar y la continuación se introduce en el contexto mediante **pure**.
- Caso (1).** Si solo queda un enlace $p \leftarrow e$, el resultado de e se transforma mediante $\langle \$ \rangle$ para producir la continuación. Cuando la continuación coincide exactamente con el patrón p , el algoritmo retorna e directamente y evita introducir una transformación identidad innecesaria.
- Caso (2).** Si un enlace $p \leftarrow e$ está seguido por más statements, estos pueden depender del valor asociado a p . Por ello se utiliza $\gg=$ y el resto de la secuencia se desazucara recursivamente dentro de la función que recibe dicho valor.
- Caso (3).** Si el único elemento restante es una agrupación $(l_1 \mid \dots \mid l_n)$, cada rama se convierte en un argumento independiente. Sus resultados se reciben mediante una función anónima que construye la continuación, y las expresiones de las ramas se combinan usando $\langle \$ \rangle$ y una sucesión de $\langle * \rangle$.
- Caso (4).** Si una agrupación aplicativa está seguida por otros statements, primero se desazucara ese resto para obtener una continuación que todavía produce un cómputo contextualizado. La combinación aplicativa genera entonces un cómputo anidado, por lo que **join** elimina un nivel del contexto y permite continuar con el plan.

La función auxiliar **desugar_arg** determina el patrón y la expresión correspondientes a cada rama de una agrupación. Para ello, intersecta las variables definidas por la rama con

las variables libres de la continuación. Así, una rama compuesta puede realizar varios enlaces internos, pero solo entrega al exterior los resultados que serán utilizados posteriormente. Luego, su propia secuencia se procesa recursivamente mediante **desugar**, lo que permite conservar dependencias monádicas dentro de una rama y combinarlas aplicativamente con las demás.

2.6.3. Aplicación del algoritmo

Para ilustrar ambas etapas se retomará el modelo probabilístico del estudiante presentado en el Código 1.1. Con el fin de concentrar la derivación en la estructura del programa, las cuatro expresiones monádicas del lado derecho de los enlaces se representarán temporalmente mediante **expr1**, **expr2**, **expr3** y **expr4**:

```
studentResults = do
  preparation <- expr1
  quiz <- expr2
  difficulty <- expr3
  finalExam <- expr4
  return (preparation, quiz, difficulty, finalExam)
```

Código 2.18: Representación abreviada del modelo probabilístico para aplicar **ApplicativeDo**.

Estas expresiones son metavariables y no definiciones independientes de Haskell: **expr2** conserva el uso libre de **preparation**, mientras que **expr4** conserva los usos de **preparation** y **difficulty**. Por su parte, **expr1** y **expr3** no utilizan variables definidas por otros statements del bloque. Denotando los cuatro statements por s_1 , s_2 , s_3 y s_4 , respectivamente, la entrada abstracta de *rearrange* es:

$$\text{do } \{s_1; s_2; s_3; s_4\} \text{ (preparation, quiz, difficulty, finalExam)}.$$

Expresión 2.4: Entrada abstracta de **rearrange** para el modelo probabilístico del estudiante.

La llamada inicial a **segments** examina los tres posibles puntos de corte. El corte después de s_1 no es válido porque **preparation** se utiliza en s_2 y s_4 . El corte después de s_2 tampoco lo es, pues s_4 todavía utiliza **preparation**. Finalmente, el corte después de s_3 separaría las definiciones de **preparation** y **difficulty** de sus usos en s_4 . En consecuencia, los cuatro statements forman un único segmento y se aplica **split**.

Para las separaciones $i \in \{1, 2, 3\}$, las llamadas recursivas a **rearrange** producen los siguientes planes y costos:

$$\begin{aligned}
i = 1 & \implies \{s_1; (s_2 \mid \{s_3; s_4\})\} & C = 3, \\
i = 2 & \implies \{s_1; s_2; s_3; s_4\} & C = 4, \\
i = 3 & \implies \{(\{s_1; s_2\} \mid s_3); s_4\} & C = 3.
\end{aligned}$$

Expresión 2.5: Planes y costos obtenidos por **rearrange** para las tres separaciones posibles.

La primera y la tercera alternativa tienen el mismo costo mínimo. Para continuar la derivación se toma la primera, que coincide con el plan producido por **GHC** para el orden original, mostrado también en la Tabla 1.2. Al sustituir nuevamente los patrones de los statements, el objeto entregado a la etapa de *desugaring* tiene la siguiente estructura intermedia:

```
do {
  preparation <- expr1;
  (quiz <- expr2 |
   { difficulty <- expr3;
     finalExam <- expr4 })
} (preparation, quiz, difficulty, finalExam)
```

Código 2.19: Plan intermedio producido por la etapa de *rearrange*.

El primer enlace permanece secuencial porque los dos componentes posteriores necesitan **preparation**. Después de obtener ese valor, la expresión asociada al **quiz** puede combinarse aplicativamente con la secuencia que obtiene la dificultad y, a partir de ella, el resultado del examen final. Al aplicar las reglas de *desugaring*, se obtiene primero la expresión abreviada:

```
studentResults =
  expr1 >>= \preparation ->
    (\quiz (difficulty, finalExam) ->
      (preparation, quiz, difficulty, finalExam))
  <$> expr2
  <*> (expr3 >>= \difficulty ->
      (\finalExam -> (difficulty, finalExam))
      <$> expr4)
```

Código 2.20: Resultado del desazucarado expresado mediante **expr1–expr4**.

En esta expresión, **expr2** se evalúa dentro del alcance de **preparation**. La segunda rama conserva un enlace monádico entre **expr3** y **expr4**, ya que la última requiere **difficulty**; el par producido por esa rama permite entregar ambos valores a la función ubicada a la izquierda de (**<\$>**). Este caso no requiere **join**, porque la agrupación aplicativa es el último componente del plan.

Finalmente, al reemplazar cada metavariable por la expresión correspondiente del programa original, se obtiene la forma explícita generada conceptualmente por las dos etapas de **ApplicativeDo**:

```

studentResults =
  Dist.uniform [Low, High] >=> \preparation ->
    (\quiz (difficulty, finalExam) ->
      (preparation, quiz, difficulty, finalExam))
    <$> quizGiven preparation
    <*> (Dist.uniform [Easy, Hard] >=> \difficulty ->
      (\finalExam -> (difficulty, finalExam))
      <$> examGiven preparation difficulty)

```

Código 2.21: Desazucarado aplicativo del modelo probabilístico del estudiante.

La expresión resultante combina operaciones monádicas y aplicativas según las dependencias del programa, pero al aplanar su plan conserva el orden relativo [1, 2, 3, 4]. La posibilidad de evaluar órdenes alternativos queda fuera del algoritmo descrito y constituye la limitación que se desarrolla en el capítulo siguiente.

Capítulo 3

Problema

En este capítulo se explicará la limitación del orden sintáctico de `ApplicativeDo`. Primero se establecerá por qué preservar el orden escrito es necesario para mantener la corrección sobre una mónada arbitraria. Luego se retomará el ejemplo probabilístico de la Sección 1.2 para mostrar que, bajo una hipótesis de conmutatividad, un orden alternativo puede exponer un plan de menor costo. Finalmente, se formulará la oportunidad que surge de esta diferencia entre el caso general y el caso conmutativo.

3.1. Limitación del Orden Sintáctico de `ApplicativeDo`

La implementación estándar de `ApplicativeDo` transforma bloques `do` preservando el orden relativo de los statements escritos por el programador. Esta restricción es correcta para mónadas arbitrarias: si un bloque contiene efectos observables, como escrituras por pantalla, lectura de archivos o mutación de estado, intercambiar dos acciones puede cambiar el comportamiento del programa.

El costo de esa seguridad es que el algoritmo solo puede descubrir paralelismo cuando este aparece compatible con el orden sintáctico original. Si dos subcómputos independientes no están ubicados en posiciones convenientes, `ApplicativeDo` puede generar un plan mejor que la ejecución secuencial, pero no necesariamente el mejor plan disponible bajo una hipótesis más fuerte. En particular, para una mónada conmutativa, ciertos statements independientes pueden cambiar de posición sin alterar la semántica. Esa posibilidad no está explotada por el algoritmo base de Marlow et al., porque su diseño debe seguir siendo válido para cualquier mónada [19].

La limitación estudiada en esta memoria aparece precisamente en esa diferencia: preservar el orden es necesario en el caso general, pero puede ser demasiado conservador cuando el programa declara explícitamente que el bloque se evalúa en una mónada conmutativa. El problema consiste entonces en determinar qué reordenamientos son semánticamente admisibles y si alguno de ellos permite que `ApplicativeDo` construya un plan de menor costo.

3.2. Evidencia del Problema y Oportunidad

Para evidenciar la limitación anterior se retoma el modelo probabilístico del estudiante presentado en el Código 1.1. La Sección 2.6.3 aplicó el algoritmo de **ApplicativeDo** a su orden original $[1, 2, 3, 4]$ y obtuvo un plan de costo 3. En esta subsección se conserva el mismo programa y se estudia qué ocurre al intercambiar únicamente s_2 y s_3 .

Se utilizarán nuevamente las metavariables **expr1**, **expr2**, **expr3** y **expr4** introducidas en el Código 2.18. Estas mantienen las variables libres de las expresiones originales, pero permiten concentrar el análisis en la estructura del bloque. Después del intercambio, el programa abreviado queda en el orden $[1, 3, 2, 4]$:

```
studentResults = do
  preparation <- expr1
  difficulty <- expr3
  quiz <- expr2
  finalExam <- expr4
  return (preparation, quiz, difficulty, finalExam)
```

Código 3.1: Representación abreviada del modelo probabilístico con los statements **s2** y **s3** intercambiados.

Este orden conserva las precedencias del programa. El statement s_1 permanece antes de s_2 y s_4 , mientras que s_3 permanece antes de s_4 , tal como muestra la Figura 1.2. Por lo tanto, el intercambio no separa ninguna definición de sus usos. Además, en el bloque no aparecen escrituras, mutación de estado ni otros efectos observables sensibles al orden: adelantar la elección de la dificultad no modifica las relaciones condicionales modeladas por **quizGiven** y **examGiven**. En concordancia con el carácter conmutativo supuesto para la mónada de distribuciones, se adopta entonces la hipótesis de que este orden es semánticamente equivalente al original.

La entrada abstracta de **rearrange** para el nuevo orden es:

$$\text{do } \{s_1; s_3; s_2; s_4\} (\text{preparation}, \text{quiz}, \text{difficulty}, \text{finalExam}).$$

Expresión 3.1: Entrada abstracta de **rearrange** para el orden $[1, 3, 2, 4]$.

La llamada inicial a **segments** tampoco encuentra un corte exterior para esta secuencia. Después de s_1 , la variable **preparation** todavía es requerida por s_2 y s_4 . Después de s_3 , tanto **preparation** como **difficulty** son utilizadas en el sufijo. Finalmente, después de s_2 , ambas variables siguen siendo necesarias para s_4 . En consecuencia, los cuatro statements forman un único segmento y **rearrange** delega la secuencia a **split**.

La función **split** evalúa las separaciones $i \in \{1, 2, 3\}$. Para cada una aplica recursivamen-

te **rearrange** al prefijo y al sufijo, y obtiene los siguientes planes de acuerdo con el modelo de costos de la Expresión 1.1:

$$\begin{aligned} i = 1 & \implies \{s_1; \{s_3; (s_2 \mid s_4)\}\} & C = 3, \\ i = 2 & \implies \{(s_1 \mid s_3); (s_2 \mid s_4)\} & C = 2, \\ i = 3 & \implies \{\{s_1; (s_3 \mid s_2)\}; s_4\} & C = 3. \end{aligned}$$

Expresión 3.2: Planes y costos obtenidos por **split** para el orden [1, 3, 2, 4].

La separación $i = 2$ divide la secuencia en el prefijo $\{s_1; s_3\}$ y el sufijo $\{s_2; s_4\}$. Al ejecutar **rearrange** sobre el prefijo, **segments** determina que s_1 y s_3 son independientes y construye $(s_1 \mid s_3)$. Una vez disponibles **preparation** y **difficulty**, la llamada sobre el sufijo también determina que s_2 y s_4 son independientes entre sí y construye $(s_2 \mid s_4)$. Cada agrupación tiene costo 1 y ambas se componen secuencialmente, por lo que el costo total es $1 + 1 = 2$. Este es el plan mínimo seleccionado por **split**.

Al sustituir los patrones de los statements, la etapa de *rearrange* entrega a *desugaring* la siguiente estructura intermedia:

```
do {
  (preparation <- expr1 |
   difficulty <- expr3);
  (quiz <- expr2 |
   finalExam <- expr4)
} (preparation, quiz, difficulty, finalExam)
```

Código 3.2: Plan intermedio producido por *rearrange* para el orden [1, 3, 2, 4].

La etapa de *desugaring* procesa primero la segunda agrupación, que es el último componente del plan, y combina **expr2** y **expr4** mediante $\langle \$ \rangle$ y $\langle * \rangle$. La primera agrupación está seguida por ese cómputo contextualizado, por lo que corresponde al caso que produce un contexto anidado. La operación **join** elimina ese nivel adicional y permite secuenciar ambas agrupaciones. El resultado abreviado es:

```

studentResults =
  join $
    (\preparation difficulty ->
      (\quiz finalExam ->
        (preparation, quiz, difficulty, finalExam))
      <$> expr2
      <*> expr4)
    <$> expr1
    <*> expr3

```

Código 3.3: Desazucarado del plan reordenado expresado mediante `expr1–expr4`.

Finalmente, al reemplazar las metavariables por las expresiones del Código 1.1, se obtiene:

```

studentResults =
  join $
    (\preparation difficulty ->
      (\quiz finalExam ->
        (preparation, quiz, difficulty, finalExam))
      <$> quizGiven preparation
      <*> examGiven preparation difficulty)
    <$> Dist.uniform [Low, High]
    <*> Dist.uniform [Easy, Hard]

```

Código 3.4: Desazucarado aplicativo del modelo probabilístico después del reordenamiento.

En comparación con el último desazucarado del orden original, presentado en el Código 2.21, la nueva expresión aumenta de una a dos apariciones de (`<*>`). La primera combina la preparación con la dificultad y la segunda combina el quiz con el examen final. Este mayor uso de la estructura applicativa concuerda con el plan de costo 2: el reordenamiento expone dos pares de cómputos independientes donde el orden original solo permitía una agrupación applicativa exterior. Por consiguiente, existe una oportunidad de mejora que el algoritmo estándar no puede aprovechar mientras conserve exclusivamente el orden escrito por el programador.

3.3. Formulación

El algoritmo de `ApplicativeDo` transforma correctamente programas sobre una mónada arbitraria. Como sus efectos pueden ser observables y sensibles al orden, preserva la secuencia relativa de los statements escritos por el programador. Esta decisión permite introducir combinadores aplicativos sin cambiar el comportamiento que tendría el desazucarado monádico original.

La situación es menos restrictiva cuando la mónada es conmutativa. Si dos statements son independientes, su intercambio puede preservar tanto las dependencias del programa como

su significado. La evidencia de la subsección anterior muestra que esta libertad semántica no es solo una propiedad abstracta: un orden equivalente puede permitir que el mismo algoritmo de `ApplicativeDo` produzca un plan de menor costo que el obtenido sobre el orden original.

Sin embargo, la implementación actual de `ApplicativeDo` en `GHC` no incorpora un tratamiento especializado para este caso: aun cuando la mónada pueda tratarse como conmutativa, no estudia órdenes alternativos de los statements antes de construir el plan. El problema abordado por esta memoria consiste, por tanto, en determinar cómo aprovechar la conmutatividad para ampliar el espacio de planes evaluables, considerando únicamente reordenamientos que preserven las dependencias del bloque, sin alterar el comportamiento general requerido para mónadas arbitrarias.

Capítulo 4

Solución

Este capítulo presenta la extensión experimental desarrollada para ampliar el espacio de planes que **ApplicativeDo** puede evaluar sobre bloques **do** declarados como conmutativos. Primero se expone el enfoque conceptual y las condiciones que mantienen el comportamiento estándar de **GHC** para los demás bloques. Luego se desarrolla el diseño algorítmico basado en dependencias, ordenamientos topológicos y evaluación de candidatos. Finalmente, se describe su implementación dentro del **Renamer**, desde la activación mediante **QualifiedDo** hasta la selección automática o forzada del árbol de statements que continúa por el flujo normal del compilador.

4.1. Enfoque Propuesto

La brecha formulada en la Sección 3.3 motiva ampliar los órdenes que **ApplicativeDo** recibe como entrada, sin reemplazar su algoritmo ni modificar el tratamiento de mónadas arbitrarias. La extensión desarrollada incorpora esta exploración únicamente cuando el programa declara un bloque con *do-notation* puede tratarse bajo la hipótesis de conmutatividad, explorada en la Sección 2.4.

Esta declaración debe ser explícita y local al bloque. Un bloque **do** ordinario continúa usando solo el orden escrito por el programador, mientras que un bloque marcado mediante la notación conmutativa queda habilitado para explorar órdenes alternativos. De este modo, la generación de estos reordenamientos se puede activar o desactivar desde el programa sin cambiar globalmente el tratamiento de otros bloques ni exigir que el compilador intente demostrar una propiedad semántica de la mónada.

Sobre un bloque **do** habilitado, la solución identifica primero las dependencias locales entre sus statements. Estas dependencias determinan qué precedencias deben conservarse y, por tanto, qué reordenamientos mantienen la semántica del programa. A partir de ellas se generan todos los reordenamientos que preservan dichas precedencias. Dentro del alcance experimental de esta memoria, la hipótesis declarada de conmutatividad permite considerar equivalentes los intercambios entre cómputos; la ausencia de una dependencia local, por sí

sola, no sería suficiente para autorizar el intercambio sobre una mónada arbitraria.

Cada reordenamiento se entrega como input al algoritmo existente de **ApplicativeDo**. Como se explicó en la Sección 2.6.1, este algoritmo conserva el orden de statements recibido, representando estos mediante una combinación de secuencias monádicas y agrupaciones aplicativas. La extensión desarrollada compara los árboles resultantes del proceso de *rearrange* mediante el modelo de costo de la Expresión 1.1 y selecciona el primer candidato que posea el menor costo entre los arboles generados. Luego, el *pipeline* del algoritmo de **ApplicativeDo** continúa en la etapa de *desaugaring* recibiendo el árbol generado por el reordenamiento óptimo con costo menor. Por consiguiente, la contribución amplía el espacio de búsqueda, mientras que **ApplicativeDo** sigue siendo responsable de construir el plan para cada orden. La Figura 4.1 resume este proceso:

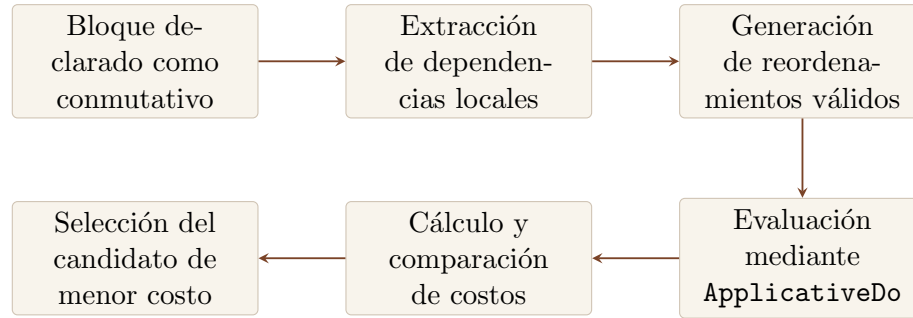


Figura 4.1: Pipeline conceptual de la extensión propuesta.

El enfoque separa así dos responsabilidades. El programador afirma que el orden de efectos independientes puede intercambiarse en el bloque `do` marcado. El compilador, por su parte, restringe la exploración a reordenamientos que conservan las relaciones locales de definición y uso de variables, evalúa todos los planes generados por la etapa de *rearrange* y selecciona el menos costoso, manteniendo sin modificaciones la compilación de los bloques `do` que no son declarados como conmutativos.

4.2. Diseño Algorítmico

La solución propuesta transforma una secuencia de statements dentro de un bloque `do` en un conjunto de reordenamientos candidatos evaluables mediante dos etapas. Primero representa las restricciones *Read-after-Write* del bloque mediante un grafo de precedencia, tal como se mostró con el ejemplo del alumno y los resultados de sus evaluaciones en la Figura 1.1. Luego enumera los ordenamientos topológicos de ese grafo, es decir, aquellos que mantienen válidas las dependencias entre statements. Finalmente, aplica la generación del plan de **ApplicativeDo** a cada reordenamiento y compara sus representaciones internas siendo estos árboles paramétricos. Esta sección define dichas etapas sin depender todavía de los tipos concretos utilizados por **GHC**.

4.2.1. Modelo de Dependencias

Antes de construir el grafo de precedencia es necesario determinar qué relaciones entre statements obligan a conservar su orden relativo. Considérense dos statements s_i y s_j tales que $i < j$. En un modelo general de asignaciones, cada uno posee un conjunto R de variables leídas y un conjunto W de variables escritas. A partir de estos conjuntos se distinguen tres clases de dependencia: *Read-after-Write* (RAW), *Write-after-Read* (WAR) y *Write-after-Write* (WAW):

$$\begin{aligned}\text{RAW}(i, j) &\iff W_i \cap R_j \neq \emptyset, \\ \text{WAR}(i, j) &\iff R_i \cap W_j \neq \emptyset, \\ \text{WAW}(i, j) &\iff W_i \cap W_j \neq \emptyset.\end{aligned}$$

Expresión 4.1: Dependencias de escritura y lectura de variables entre dos statements s_i y s_j .

Una dependencia RAW establece una relación productor–consumidor: s_j lee una variable cuyo valor fue escrito por s_i . Una dependencia WAR aparece cuando s_i debe leer una variable antes de que s_j la sobrescriba; invertir ambos statements haría que la lectura observara el nuevo valor. Finalmente, una dependencia WAW relaciona dos escrituras sobre una misma variable y obliga a conservar cuál de ellas determina su valor posterior. Estas dos últimas relaciones presuponen que distintas escrituras pueden actuar sobre una misma identidad mutable, como ocurre en un modelo imperativo de asignaciones. Alterar cualquiera de estas precedencias puede modificar los valores observados o el estado final del programa [2, 12].

La *do-notation* de Haskell presenta un comportamiento diferente. En un statement de la forma $\mathbf{p} \leftarrow \mathbf{e}$, el patrón $\mathbf{p}$ introduce una nueva vinculación léxica, denominada *binder*; no actualiza destructivamente una variable definida con anterioridad. Si un statement posterior reutiliza el mismo identificador textual, introduce otro binder que oculta al primero en su nuevo alcance. Este comportamiento concuerda con la pureza descrita en la Sección 2.1 y con las reglas de alcance de la *do-notation* [16, sección 3.14].

El Renamer de GHC hace explícita esta diferencia al resolver los identificadores del programa. Cada binder recibe un **Name** único y cada uso queda asociado con la identidad que se encuentra en alcance en ese punto. Por ello, dos definiciones escritas como $\mathbf{x}$ en el programa fuente no representan dos escrituras internas sobre el mismo nombre.

El Código 4.1 ilustra este comportamiento con un bloque **do** de tipo **Maybe Int**. En el programa fuente, $\mathbf{y}$ lee la primera vinculación de $\mathbf{x}$ antes de que un statement posterior vuelva a utilizar ese identificador. La columna derecha presenta esquemáticamente las identidades diferenciadas durante el renombrado:

<pre>do x <- Just 2 y <- Just ((2*) x) x <- Just 3 return (x + y)</pre>	$\Rightarrow$	<pre>do x_a1T2 <- Just 2 y_a1T3 <- Just ((2*) x_a1T2) x_a1T4 <- Just 3 return (x_a1T4 + y_a1T3)</pre>
--	---------------	--

Código 4.1: Representación esquemática de los binders antes y después del renombrado.

La primera y la tercera definición escriben, respectivamente, los nombres `x_a1T2` y `x_a1T4`. Como son identidades diferentes, sus conjuntos de escritura no se intersectan y, por tanto, no existe una dependencia WAW. De manera análoga, el segundo statement lee `x_a1T2`, mientras que el tercero escribe `x_a1T4`; la intersección entre esa lectura y la escritura posterior también es vacía, por lo que no existe una dependencia WAR. La relación local efectiva entre los statements reordenables es RAW: la definición de `x_a1T2` debe permanecer antes del statement que define `y_a1T3`. En consecuencia, el grafo de precedencia utilizado por la solución puede especializarse solo con dependencias RAW entre nombres renombrados.

El conjunto de variables libres que entrega el Renamer para un statement contiene todos los `Names` utilizados que no son introducidos por ese mismo statement. Esto incluye tanto referencias a binders definidos por statements anteriores como nombres globales, entre ellos funciones, operadores y constructores de datos. Por ejemplo, en el segundo statement del Código 4.1 aparecen como nombres libres `x_a1T2`, el constructor `Just` y el operador `(*)`, pero solo el primero corresponde a un valor producido dentro del bloque `do`. Para distinguir las lecturas locales de las referencias externas se definen los siguientes conjuntos para cada statement s_i :

$$\begin{aligned}
\text{WRITE}(s_i) &= \text{binders}(s_i), \\
\text{ALL_WRITES} &= \bigcup_{k=1}^n \text{WRITE}(s_k), \\
\text{READ}_{\text{local}}(s_i) &= \text{FV}(s_i) \cap \text{ALL_WRITES}.
\end{aligned}$$

Expresión 4.2: Escrituras y lecturas locales de un statement renombrado.

El conjunto $\text{WRITE}(s_i)$ contiene los `Names` de los binders introducidos por s_i , mientras que ALL_WRITES reúne todos los nombres producidos por los statements del bloque `do`. Por su parte, $\text{FV}(s_i)$ conserva el conjunto completo de nombres libres informado por el Renamer. Al intersectarlo con ALL_WRITES , $\text{READ}_{\text{local}}(s_i)$ retiene únicamente los nombres cuyo valor se origina por un binder de un statement anterior. En el ejemplo del Código 4.1, `x_a1T2` permanece como lectura local del segundo statement, mientras que `Just` y `(*)` son descartados porque no pertenecen a ALL_WRITES , sino que a $\text{FV}(s_2)$.

Con esta información, la restricción relevante entre dos statements s_i y s_j queda dada por

$$i \longrightarrow j \iff i < j \wedge \text{WRITE}(s_i) \cap \text{READ}_{\text{local}}(s_j) \neq \emptyset.$$

Expresión 4.3: Dependencia RAW local utilizada por la extensión.

La condición $i < j$ orienta la dependencia según el orden del programa original. La intersección no vacía indica que existe al menos un **Name** definido por s_i y utilizado localmente por s_j ; dicho nombre actúa como testigo de una relación productor–consumidor que debe preservarse. Debido a que la comparación se realiza sobre nombres renombrados, la arista no puede surgir solo por la reutilización de un mismo identificador textual.

Estas definiciones reducen las restricciones semánticas del bloque a un conjunto de relaciones RAW locales entre statements. A partir de ellas, la etapa siguiente construye el grafo de precedencia cuyos vértices representan statements y cuyas aristas representan las dependencias que todo reordenamiento válido debe conservar.

4.2.2. Construcción del Grafo de Precedencia

Sea $G = (V, E)$ el grafo de precedencia del bloque y sea n la cantidad de statements que participan en el reordenamiento. Cada vértice de $V = \{1, \dots, n\}$ representa uno de estos statements según su posición original, y cada arista $(i, j) \in E$ representa la dependencia RAW local de la Expresión 4.3. El statement terminal que entrega el resultado del bloque permanece fijo y no participa en las permutaciones, de la misma manera que la expresión final se mantiene fuera de la entrada de *rearrange* en la presentación de Marlow et al.

El Código 4.2 presenta el algoritmo abstracto de construcción. Primero calcula la información local de cada statement y luego compara todos los pares que respetan el orden original.

```

buildPrecedenceGraph(statements):
  n := length(statements)
  allWrites := union(write(s) for s in statements)
  graph := graph with vertices {1, ..., n}

  for each statement s_i:
    writesLocal[i] := write(s_i)
    readsLocal[i] := intersection(freeVars(s_i), allWrites)

  for i := 1 to n:
    for j := i + 1 to n:
      if intersection(writesLocal[i], readsLocal[j]) != empty:
        add edge i -> j to graph

  return graph

```

Código 4.2: Pseudocódigo para construir el grafo de precedencia RAW.

La condición $i < j$ cumple dos funciones. Primero, expresa que el grafo conserva una precedencia presente en el programa original. Segundo, garantiza su aciclicidad: los índices aumentan estrictamente a lo largo de toda arista, por lo que ningún camino puede regresar a su nodo inicial. El algoritmo realiza

$$\sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$$

comparaciones entre pares. Por tanto, la cantidad de pruebas de dependencia es $O(n^2)$, sin incluir el costo interno de construir e intersectar los conjuntos de nombres. Para el ejemplo probabilístico del Código 1.1, la aplicación de este algoritmo produce las aristas $1 \rightarrow 2$, $1 \rightarrow 4$ y $3 \rightarrow 4$. El grafo ya fue presentado en la Figura 1.1.

4.2.3. Enumeración de Reordenamientos Válidos

Un reordenamiento de statements respeta las restricciones RAW del grafo si, para cada arista $i \rightarrow j$, el statement s_i aparece antes que s_j . Una secuencia que contiene todos los vértices y satisface esta condición se denomina ordenamiento topológico. Esta subsección presenta primero el algoritmo de Kahn para obtener uno de estos órdenes y luego desarrolla la variante ramificada utilizada para enumerarlos todos.

Algoritmo de Kahn

El algoritmo de Kahn obtiene un ordenamiento topológico de un grafo dirigido acíclico a partir del grado de entrada de sus vértices [14]. El grado de entrada de un vértice corresponde a la cantidad de aristas que llegan hasta él. Por tanto, un vértice con grado de entrada cero no posee predecesores pendientes y puede ocupar la siguiente posición del ordenamiento.

El procedimiento calcula primero los grados de entrada y agrega a una cola todos los vértices cuyo valor es cero. El uso de esta cola determina una estrategia de procesamiento en amplitud, análoga a BFS. Luego extrae el primer vértice de la cola, lo incorpora al orden parcial y elimina conceptualmente sus aristas salientes. Cada eliminación reduce en uno el grado de entrada del sucesor correspondiente; si alguno alcanza cero, se agrega al final de la cola. El proceso continúa hasta vaciarla. En un DAG se emiten todos los vértices, mientras que una cola vacía antes de completar el orden indica la existencia de un ciclo. Como cada vértice se incorpora una vez y cada arista se examina al eliminarla, el costo del algoritmo es $O(|V| + |E|)$.

En el grafo del ejemplo probabilístico, presentado en la Figura 1.1, los grados de entrada iniciales de s_1 , s_2 , s_3 y s_4 son, respectivamente, 0, 1, 0 y 2. Al insertar los vértices iniciales según el índice del statement, la cola comienza como $[s_1, s_3]$. La Tabla 4.1 resume la ejecución completa.

Paso	Vértice emitido	Cola actualizada	Orden parcial
0	—	[s1,s3]	[]
1	s ₁	[s3,s2]	[1]
2	s ₃	[s2,s4]	[1,3]
3	s ₂	[s4]	[1,3,2]
4	s ₄	[]	[1,3,2,4]

Tabla 4.1: Aplicación del algoritmo de Kahn al grafo del ejemplo probabilístico.

Cuando se emite s_1 , el grado de s_2 se vuelve cero, pero este vértice se agrega detrás de s_3 , que ya se encontraba en la cola. De esta manera se obtiene el orden $[1, 3, 2, 4]$. Esta secuencia coincide con la disposición del mismo grafo de precedencia mostrada en la Figura 1.2, donde todas las aristas conservan su orientación hacia statements posteriores.

Enumeración Exhaustiva

El algoritmo clásico de Kahn selecciona la cabeza de la cola y produce un único ordenamiento. La solución propuesta debe obtener todas las alternativas, por lo que reemplaza esa elección única por una ramificación sobre cada vértice no emitido cuyo grado de entrada sea cero. Cada rama emite uno de ellos, actualiza los grados de sus sucesores y repite el procedimiento sobre el estado resultante. Conceptualmente, esto equivale a ejecutar el mismo criterio sobre el grafo reducido después de retirar el vértice seleccionado.

El Código 4.3 presenta este procedimiento. El conjunto **emitted** identifica los vértices ya retirados y **indegree** conserva los grados correspondientes al grafo reducido. La lista **order** acumula el prefijo del reordenamiento construido por cada rama.

```

enumerateTopologicalOrders(graph, indegree, emitted, order):
    if length(order) == numberOfVertices(graph):
        emit order
        return

    choices := every non-emitted vertex v with indegree[v] == 0

    for each v in choices:
        nextIndegree := copy(indegree)
        for each successor w of v:
            nextIndegree[w] := nextIndegree[w] - 1

        enumerateTopologicalOrders(
            graph,
            nextIndegree,
            union(emitted, {v}),
            order ++ [v])

```

Código 4.3: Pseudocódigo para enumerar los ordenamientos topológicos del grafo.

Cada rama agrega únicamente un vértice cuyos predecesores ya fueron emitidos. Por inducción sobre la longitud de la lista acumulada, toda secuencia producida respeta las aristas. A la inversa, toda primera posición de un ordenamiento topológico debe tener grado de entrada cero; como el algoritmo ramifica sobre todas esas posibilidades y repite el argumento sobre el grafo restante, también alcanza cada orden válido.

La enumeración está necesariamente condicionada por la cantidad de resultados. Si P es el número de ordenamientos topológicos, el algoritmo debe producir P listas de longitud n ; la implementación concreta las conserva para evaluarlas posteriormente. En el caso extremo con un grafo de precedencia sin aristas, todos los statements son independientes respecto a la dependencia RAW y la cantidad de listas o reordenamientos es $n!$. Por ende en el peor de los casos cuando se quieren obtener todos los posibles reordenamientos de una secuencia de statements completamente independiente entre ellos, el orden del algoritmo termina siendo $n!$.

La Figura 4.2 desarrolla la enumeración para el ejemplo probabilístico cuyo grafo se muestra en la Figura 1.1. Se recorren las alternativas disponibles por índice creciente para reproducir el orden estable de los candidatos. Cada estado muestra el prefijo emitido π y el subgrafo $G[R]$ inducido por el conjunto R de vértices restantes. Una arista del árbol representa la emisión del statement indicado y la llamada recursiva resultante.

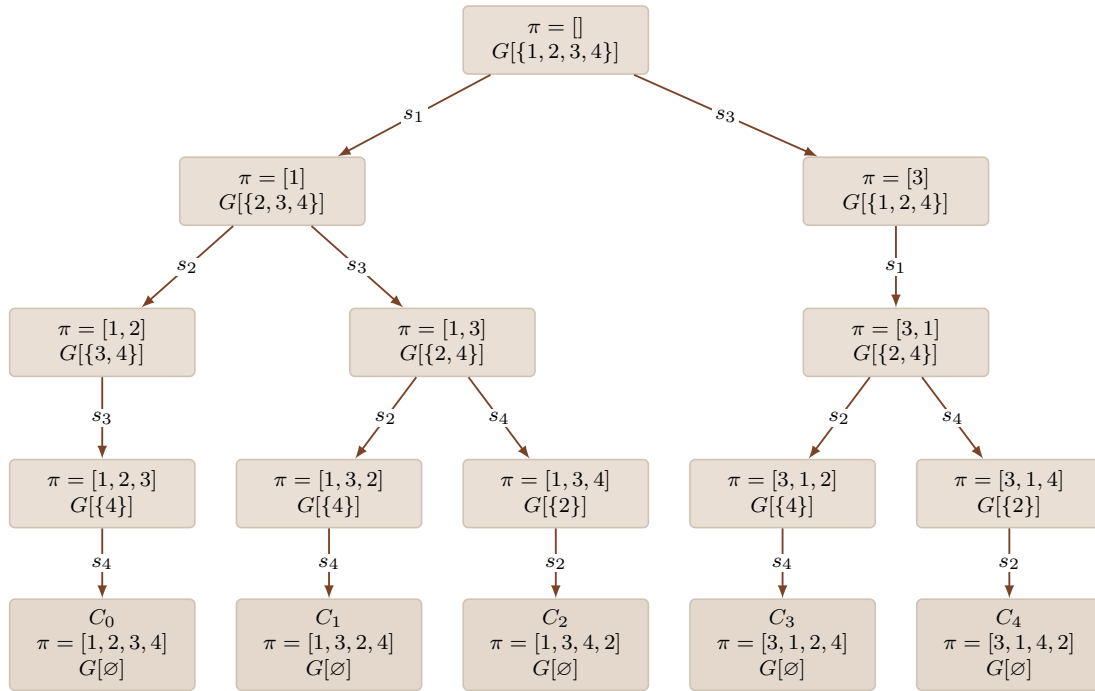


Figura 4.2: Árbol de llamadas para enumerar los reordenamientos topológicos del ejemplo probabilístico. La raíz representa la invocación inicial y los otros 15 estados corresponden a llamadas recursivas; las cinco hojas resaltadas son los candidatos obtenidos.

Cada camino desde la raíz hasta una hoja contiene los cuatro statements en un orden que respeta las dependencias. La Tabla 4.2 reúne los cinco reordenamientos. El candidato cero reproduce el orden original; los demás intercalan los statements independientes sin invertir ninguna de las tres aristas del grafo.

Candidato	Reordenamiento
0	[1, 2, 3, 4]
1	[1, 3, 2, 4]
2	[1, 3, 4, 2]
3	[3, 1, 2, 4]
4	[3, 1, 4, 2]

Tabla 4.2: Reordenamientos topológicos válidos para el ejemplo probabilístico.

4.2.4. Evaluación y Selección de Candidatos

La tercera etapa aplica el mismo algoritmo de **ApplicativeDo** a cada orden de la Tabla 4.2. La extensión no introduce una estrategia alternativa para agrupar statements: cada permutación se convierte en una nueva secuencia de entrada para *rearrange*, que decide dónde utilizar composición monádica y dónde construir agrupaciones aplicativas. El desazucarado posterior tampoco cambia y continúa según los casos descritos en la Sección 2.6.2.

El resultado de evaluar un orden se modela como un candidato compuesto por su índice, la permutación, el árbol de statements y el costo de ese árbol. Si el bloque genera P permutaciones y el algoritmo de **ApplicativeDo** elegido cuesta $A(n)$ para una entrada de n statements, esta fase requiere P evaluaciones y tiene un costo global proporcional a $P A(n)$. La implementación de **GHC** dispone de una variante heurística $O(n^2)$ y una variante óptima basada en programación dinámica $O(n^3)$ para construir cada árbol.

La política automática escoge el primer candidato que alcanza el menor costo entre los árboles producidos por la variante seleccionada de **ApplicativeDo**. Este criterio entrega un desempate determinista sin introducir una preferencia semántica adicional entre órdenes equivalentes dentro del modelo. Cuando **OptimalApplicativeDo** está habilitado, cada árbol es además el óptimo encontrado por programación dinámica para su orden; en caso contrario se utiliza la construcción heurística. En el ejemplo, el candidato cero conserva el costo 3 de **ApplicativeDo** sobre el orden original, mientras que los candidatos uno a cuatro alcanzan costo 2. Por ello se selecciona el candidato uno, cuyo plan $(s_1 \mid s_3); (s_2 \mid s_4)$ ya fue comparado con los planes anteriores en la Tabla 1.2.

4.3. Implementación en GHC

El diseño anterior se implementó directamente en **GHC**, dentro del procesamiento de expresiones del Renamer. Esta ubicación permite reutilizar la representación del compilador, la información de nombres libres y el algoritmo existente de **ApplicativeDo**, sin introducir una herramienta externa de reescritura ni operar sobre una representación que ya haya perdido la estructura de la *do-notation*.

4.3.1. Ubicación en el Pipeline del Compilador

Las opciones de línea de comandos y los pragmas del módulo configuran, entre otros aspectos, las extensiones de lenguaje que utilizarán las etapas posteriores. A partir de esa configuración, el Parser construye el árbol de sintaxis; el Renamer resuelve los identificadores y sus alcances; el Typechecker verifica e incorpora información de tipos; y el Desugarer traduce la representación tipada hacia Core. Para una expresión, la progresión de fases puede resumirse como:

$$\begin{array}{c} \text{código fuente} \xrightarrow{\text{Parser}} \text{HsExpr GhcPs} \xrightarrow{\text{Renamer}} \text{HsExpr GhcRn} \\ \xrightarrow{\text{Typechecker}} \text{HsExpr GhcTc} \xrightarrow{\text{Desugarer}} \text{Core}. \end{array}$$

Expresión 4.4: Fases relevantes del AST de una expresión en GHC.

El AST de GHC está parametrizado por la fase de compilación. Los parámetros `GhcPs`, `GhcRn` y `GhcTc` distinguen, respectivamente, los nodos parseados, renombrados y verificados por tipos. Una expresión de *do-notation* se representa mediante el constructor `HsDo`, que contiene el tipo de bloque y su lista de statements. El Renamer especializa el procesamiento según el constructor de cada expresión y transforma un nodo `HsDo` parseado en su versión renombrada mediante el caso resumido en el Código 4.4.

```
rnExpr (HsDo _ do_or_lc (L l stmts))
= do { ((stmts1, _), fvs1) <-
      rnStmtsWithFreeVars (HsDoStmt do_or_lc) rnExpr stmts
      (\ _ -> return ((), emptyFVs))
  ; (pp_stmts, fvs2) <-
      postProcessStmtsForApplicativeDo do_or_lc stmts1
  ; return (HsDo noExtField do_or_lc (L l pp_stmts),
            fvs1 'plusFV' fvs2) }
```

Código 4.4: Flujo de renombrado de una expresión `HsDo` en `GHC.Rename.Expr`.

La función `rnStmtsWithFreeVars` retorna cada statement renombrado junto con las variables libres de su cuerpo. `postProcessStmtsForApplicativeDo` comprueba luego que `ApplicativeDo` esté habilitado, que el contexto corresponda a `DoExpr` y que el código no se encuentre dentro de un bracket de Template Haskell. `rearrangeForApplicativeDo` se invoca solo después de estas comprobaciones. La extensión se incorporó en esta última función, antes de que `stmtTreeToStmts` reconstruya la lista con nodos `ApplicativeStmt`.

La elección del Renamer permite, por tanto, trabajar sobre statements que ya poseen identidades únicas y todavía conservan su estructura fuente. Un análisis posterior sobre Core habría requerido reconstruir información de la *do-notation*; uno anterior al renombrado no dispondría de las identidades necesarias para distinguir correctamente binders con la misma escritura textual.

4.3.2. Activación Declarativa mediante `QualifiedDo`

`QualifiedDo` permite escribir un bloque como `M.do` y resolver desde el módulo calificador `M` las operaciones utilizadas por la notación, en vez de usar necesariamente las operaciones habituales de `Prelude` [10]. La implementación aprovecha esta sintaxis existente para realizar un *opt-in* visible en cada bloque. El Código 4.5 muestra su forma general.

```
{-# LANGUAGE ApplicativeDo #-}
{-# LANGUAGE QualifiedDo #-}

import qualified Control.Monad.CommutativeDo as CD

pair :: CD.CommutativeMonad m => m a -> m b -> m (a, b)
pair ma mb = CD.do
  a <- ma
  b <- mb
  CD.return (a, b)
```

Código 4.5: Bloque activado mediante `Control.Monad.CommutativeDo`.

Para soportar esta notación se agregaron dos módulos. `GHC.Internal.Control.Monad.CommutativeDo`, dentro de `ghc-internal`, define la clase marcador `CommutativeMonad`, el valor `__commutative_do__` y las operaciones básicas de `QualifiedDo`, como `(>>=)`, `(>>)` y `fail`. También exporta `return`, `pure`, `fmap`, `(<*>)` y `join`, necesarias cuando el procesamiento de `ApplicativeDo` transforma el bloque. El módulo público `Control.Monad.CommutativeDo`, dentro de `base`, reexporta esa interfaz para los programas de usuario.

La clase vacía `CommutativeMonad` representa una afirmación del programador, no una demostración realizada por GHC. Las funciones exportadas por el módulo `CD` incorporan esa restricción de clase y su uso será verificado durante el typechecking. De manera independiente, durante el renombrado `isCommutativeQualifiedDo` consulta únicamente si el módulo que califica el bloque exporta un nombre `__commutative_do__`. La presencia del marcador habilita la generación de órdenes alternativos; su ausencia conserva el orden original como único candidato. Por ello, el mecanismo completo sigue siendo una declaración de confianza y no una prueba interna de conmutatividad.

La activación efectiva requiere además que `ApplicativeDo` esté habilitado y que el bloque alcance el procesamiento descrito en la subsección anterior. Aunque el detector contempla también la forma interna de `MDoExpr`, el gate actual de `postProcessStmtsForApplicativeDo` aplica la transformación solo sobre `DoExpr`; por ello, los experimentos utilizan `CD.do`. La flag de selección presentada al final de esta sección no participa en la activación.

4.3.3. Representación de Statements y Construcción del Grafo

La información requerida por el grafo se reúne en `StmtDepInfo`. Como muestra el Código 4.6, cada valor conserva el índice estable del statement, el nodo renombrado, sus variables

libres originales y los conjuntos locales de lectura y escritura.

```
data StmtDepInfo = StmtDepInfo
  { sdiIndex      :: !Int
  , sdiStmt       :: !(ExprLStmt GhcRn)
  , sdiFvsOriginal :: !FreeVars
  , sdiReadsLocal  :: !FreeVars
  , sdiWritesLocal :: !FreeVars
  }

type StmtDepNode = DG.Node Int StmtDepInfo
type StmtDepGraph = DG.Graph StmtDepNode
```

Código 4.6: Información de dependencias almacenada para cada statement.

El tipo `ExprLStmt GhcRn` corresponde a un statement de expresión localizado y renombrado. `sdiFvsOriginal` conserva todos los nombres libres entregados por el Renamer, incluidas referencias locales y nombres globales como funciones o constructores. `sdiReadsLocal` restringe ese conjunto a nombres definidos en el mismo bloque, mientras que `sdiWritesLocal` contiene los binders introducidos por el statement. Ambos campos usan `FreeVars`, representación basada en conjuntos de `Names`.

El Código 4.7 implementa las definiciones de la Expresión 4.2. La utilidad existente `collectStmtBinders` recorre las distintas formas de statement y, con `CollNoDictBinders`, recolecta únicamente binders de términos.

```
computeStmtsInfo stmts = zipWith mkStmtInfo [1 ..] stmts
  where
    all_writes =
      mkNameSet
        (concatMap
          (collectStmtBinders CollNoDictBinders . unLoc . fst)
          stmts)

    mkStmtInfo i (stmt, fvs) = StmtDepInfo
      { sdiIndex      = i
      , sdiStmt       = stmt
      , sdiFvsOriginal = fvs
      , sdiReadsLocal = fvs `intersectNameSet` all_writes
      , sdiWritesLocal =
          mkNameSet
            (collectStmtBinders CollNoDictBinders (unLoc stmt))
      }
```

Código 4.7: Cálculo de lecturas y escrituras locales en el Renamer.

La detección RAW se reduce a una intersección de conjuntos. `hasDependency` convierte

el conjunto resultante en el predicado usado para insertar aristas:

```
depRAW :: StmtDepInfo -> StmtDepInfo -> FreeVars
depRAW si sj =
    sdiWritesLocal si 'intersectNameSet' sdiReadsLocal sj

hasDependency :: StmtDepInfo -> StmtDepInfo -> Bool
hasDependency si sj = not (isEmptyNameSet (depRAW si sj))
```

Código 4.8: Detección de una dependencia RAW entre dos statements.

En vez de implementar desde cero una estructura de grafo, la solución utiliza `GHC.Data.Graph.Directed`. Cada `DG.DigraphNode` almacena un `StmtDepInfo`, usa el índice como clave y registra como destinos los statements posteriores que dependen de uno de sus binders. La comprensión del Código 4.9 corresponde directamente al doble recorrido del Código 4.2.

```
buildStmtDepNodes infos = map mkNode infos
  where
    mkNode si = DG.DigraphNode
      { DG.node_payload = si
      , DG.node_key = sdiIndex si
      , DG.node_dependencies =
          [ sdiIndex sj
          | sj <- infos
          , sdiIndex si < sdiIndex sj
          , hasDependency si sj
          ]
      }

buildStmtsDependencyGraph stmts =
  DG.graphFromEdgedVerticesOrd
    $ buildStmtDepNodes
    $ computeStmtsInfo stmts
```

Código 4.9: Construcción de nodos y aristas mediante `GHC.Data.Graph.Directed`.

4.3.4. Generación de Permutaciones

La función `enumerateSemanticTopSortsBounded` recibe el `StmtDepGraph` y materializa los órdenes descritos en el Código 4.3. Para ello construye un `IntMap` con los grados de entrada, obtiene los sucesores desde el grafo original y los predecesores desde su transposición. En cada llamada recursiva recorre las claves aún no emitidas, conserva aquellas cuyo grado es cero y crea una rama por elección.

La implementación no elimina físicamente los nodos. Al emitir un índice, decrementa en

una copia del mapa el grado de entrada de sus sucesores y acumula el nodo seleccionado. Al completar la cantidad original de vértices, invierte el acumulador y extrae el `StmtDepInfo` almacenado en cada nodo. El recorrido estable de las claves hace que la primera rama reproduzca el orden original, que aparece como candidato cero.

Cada resultado se transforma nuevamente en la secuencia de statements `ExprLStmt GhcRn` anotados con sus `FreeVars`, que es la entrada esperada por `ApplicativeDo`. Las variables libres originales se conservaron precisamente para no recalcular ni perder esta anotación al cambiar el orden de los statements.

4.3.5. Integración con `ApplicativeDo`

Después de separar el `LastStmt`, `rearrangeForApplicativeDo` construye el grafo y enumera las permutaciones solo si `isCommutativeQualifiedDo` retorna verdadero. En caso contrario crea una lista unitaria con el orden original. La opción `Opt_OptimalApplicativeDo` determina luego si se utiliza `mkStmtTreeHeuristic` o `mkStmtTreeOptimal`; la función elegida se aplica a cada secuencia.

La representación interna de los planes es el tipo `StmtTree` del Código 4.10. Sus tres constructores corresponden a los tres casos del modelo estructural: un statement elemental, una composición secuencial mediante *bind* y una agrupación de ramas aplicativas. La función `stmtTreeCost` implementa directamente las reglas de la Expresión 1.1.

```
data StmtTree a
  = StmtTreeOne a
  | StmtTreeBind (StmtTree a) (StmtTree a)
  | StmtTreeApplicative [StmtTree a]

stmtTreeCost :: ExprStmtTree -> Cost
stmtTreeCost (StmtTreeOne _) = 1
stmtTreeCost (StmtTreeBind l r) =
  stmtTreeCost l + stmtTreeCost r
stmtTreeCost (StmtTreeApplicative ts) =
  case ts of
    [] -> 0
    _ -> Partial.maximum (map stmtTreeCost ts)
```

Código 4.10: Representación de planes y cálculo de su costo estructural.

`StmtsPermutationInfo` asocia cada árbol con la permutación que lo originó. El índice de candidato parte desde cero, mientras que los índices de `StmtDepInfo` parten desde uno y conservan las posiciones del programa fuente.

```

data StmtPermutationInfo = StmtPermutationInfo
  { candIndex :: !Int
  , candPerm  :: [StmtDepInfo]
  , candTree  :: ExprStmtTree
  , candCost  :: !Cost
  }

let stmt_trees = map mkStmtTree all_permutations
let all_permutations_info =
  zipWith3 mkStmtPermutationInfo
    [0 :: Int ..] validStmtPerms stmt_trees
let best_candidate = case nonEmpty all_permutations_info of
  Just cs -> minimumBy (comparing candCost) cs
  Nothing -> panic "rearrangeForApplicativeDo: empty stmt_trees"

```

Código 4.11: Información conservada para evaluar y seleccionar candidatos.

Conceptualmente, el último bloque aplica un `map` para evaluar las permutaciones, combina orden y árbol mediante `zipWith3`, y usa `minimumBy` para seleccionar el primer mínimo. Un `filter` separado se utiliza en las trazas para contar cuántos candidatos empatan con el mínimo, pero no interviene en la selección automática.

Una vez elegido el candidato, `stmtTreeToStmts` reconstruye los statements finales, agrega nuevamente el statement terminal y genera `ApplicativeStmt` donde el árbol contiene agrupaciones aplicativas. Desde ese punto, el programa continúa por el typechecking y el desazucarado normales de GHC.

4.3.6. Selección Forzada para Validación

La política de costo mínimo permite evaluar el resultado optimizado, pero no basta para comprobar individualmente todas las permutaciones producidas. Con ese propósito se agregó la flag experimental

```
-fado-reorder-candidate-n=<n>.
```

`GHC.Driver.DynFlags` almacena su valor como `Maybe Int` y `GHC.Driver.Session` registra la opción de línea de comandos. `rearrangeForApplicativeDo` calcula primero el mejor candidato y luego intenta obtener el elemento indicado por la flag:

```
let selected_candidate = do
  i <- reorder_cand_n
  guard (i >= 0)
  listToMaybe (drop i all_permutations_info)

let final_candidate =
  fromMaybe best_candidate selected_candidate
```

Código 4.12: Selección opcional de un candidato por índice.

Un índice válido selecciona el candidato base cero solicitado. Si la flag no está presente, el índice es negativo o queda fuera del rango generado, se conserva `best_candidate`. Esta opción no habilita la conmutatividad, no crea candidatos adicionales y no altera sus costos; solo reemplaza la decisión final una vez completado el mismo pipeline. El capítulo siguiente utiliza este mecanismo para compilar y ejecutar cada permutación por separado durante la validación semántica.

Capítulo 5

Validación

5.1. Metodología

La validación busca comprobar dos propiedades experimentales. Primero, que las permutaciones generadas por la extensión preserven la salida observable de los programas evaluados. Segundo, que el reordenamiento pueda producir planes con menor costo abstracto que `ApplicativeDo` aplicado únicamente al orden original.

El pipeline de validación compila varias variantes de cada caso:

- el programa original;
- el programa con `ApplicativeDo` normal, sin marcador conmutativo;
- el candidato óptimo seleccionado automáticamente por la extensión;
- cada permutación válida, forzada mediante la flag de selección de candidato.

Luego, el pipeline ejecuta cada binario, captura `stdout`, registra el código de salida y compara los resultados contra la referencia. En los dos corpus sintéticos, una validación exitosa exige que las salidas y códigos de salida coincidan. En el corpus sintético probabilístico, los programas imprimen distribuciones normalizadas para evitar diferencias superficiales de representación. El caso con `IO` se mantiene separado como control negativo: su divergencia observable es el resultado esperado y muestra por qué el reordenamiento requiere una declaración explícita de conmutatividad.

Existen dos modos de ejecución. Los casos que no dependen de paquetes externos pueden compilarse directamente con el `GHC` modificado. Los casos probabilísticos se validan mediante `Cabal`, porque usan el paquete `probability`. En ambos modos, el resultado se guarda bajo `tests/`, junto con trazas del `Renamer`, resúmenes de costo y outputs de ejecución.

El ejemplo guía sigue este segundo modo. Su programa y sus resultados se encuentran, respectivamente, en:

```
experiments/main-example/main.hs
tests/main-example
```

Este caso se usa transversalmente para explicar el problema y la solución, pero no se contabiliza como un tercer corpus: la evaluación agregada continúa organizada en un corpus estructural con `Maybe` y otro probabilístico con `Dist.T Rational`.

5.2. Artefactos de Validación

La validación deja artefactos reproducibles para inspeccionar el comportamiento del compilador y respaldar los resultados. Los experimentos viven bajo `experiments/`, mientras que los resultados se guardan bajo `tests/`. Los dos corpus siguen una estructura paralela por mónada, familia, caso y variante. El ejemplo guía usa las rutas directas `experiments/main-example` y `tests/main-example`.

Los archivos más relevantes son:

Artefacto	Función
<code>summary.log</code>	Resume costos y cantidad de permutaciones.
<code>semantic-validation.log</code>	Registra compilaciones, ejecuciones y comparación final.
<code>logs/raw.log</code>	Guarda la traza cruda del Renamer.
<code>graph.log</code>	Presenta el grafo RAW en forma legible.
<code>logs/optimal-reorder.log</code>	Muestra el candidato elegido automáticamente.
<code>logs/original-ado.log</code>	Muestra <code>ApplicativeDo</code> normal, sin reordenamiento.
<code>logs/permutation-i.log</code>	Muestra el candidato <code>i</code> forzado por flag.

Estos artefactos cumplen dos roles. Primero, permiten depurar la extensión observando el grafo, las permutaciones y los árboles seleccionados. Segundo, entregan la base empírica para comparar costos y salidas entre variantes.

5.3. Métricas

La comparación de planes usa métricas emitidas por las trazas del Renamer. Las claves principales son:

```
original-cost
applicative-do-cost
reorder-and-ado-minimum-cost
generated-permutations
minimum-cost-permutations
```

La primera métrica corresponde al costo secuencial base del bloque. La segunda mide el costo obtenido por `ApplicativeDo` sobre el orden original. La tercera mide el menor costo encontrado al evaluar las permutaciones válidas. Las dos últimas indican cuántos candidatos fueron generados y cuántos de ellos empatan en el costo mínimo.

El costo es abstracto, no una medición de tiempo real. Las tres métricas siguen el modelo de costos estructural presentado. Por ello, sirven para comparar la forma del plan de ejecución, no para afirmar una aceleración empírica directa en tiempo de pared.

Además de las métricas de costo, la validación registra el resultado semántico de cada caso. Un caso exitoso termina con `RESULT: OK`; un control negativo diseñado para mostrar cambio observable puede terminar con `RESULT: FAIL`, como ocurre con el caso de `IO`.

5.4. Corpus

5.4.1. Corpus Sintético Maybe

El primer corpus sintético usa la mónada `Maybe`. Su objetivo es validar fenómenos estructurales del Renamer y del grafo de precedencia con programas pequeños, deterministas y fáciles de comparar. La descripción completa de los casos está en el archivo `cases.md` del corpus `maybe-monad`.

Las familias implementadas cubren los siguientes aspectos:

- **dependency-shapes**: formas de grafo como cadenas, grafo vacío, diamante, fanout, fanin y cadenas independientes.
- **binders**: patrones que definen múltiples variables, patrones de lista, patrones anidados, wildcards, `as-patterns` y patrones `lazy`.
- **let-statements**: interacción entre statements `let`, binders monádicos y shadowing.
- **body-stmts**: statements sin binder, tanto independientes como dependientes.
- **maybe-failure**: comportamiento de `Nothing`, fallos de patrón y guards.
- **shadowing**: reutilización del mismo nombre fuente con nombres renombrados distintos.
- **cost-selection**: casos donde el orden original no es mínimo, todos los costos empatan o existe un mínimo único.
- **controls**: controles negativos y positivos para verificar que el marcador conmutativo activa la extensión solo cuando corresponde.

Este corpus permite aislar errores en la recolección de binders, el cálculo de variables libres locales, la construcción de aristas RAW y la selección de candidatos. En particular, la familia `cost-selection` comprueba distintos comportamientos de la política de selección, aunque el caso guía transversal del informe se mantiene separado bajo `experiments/main-example`.

5.4.2. Corpus Sintético Probabilístico

El segundo corpus sintético usa la mónada probabilística `Dist.T Rational` del paquete `probability`. Este corpus conecta la extensión con el dominio principal de la memoria: programas probabilísticos finitos escritos en `Haskell`. Su descripción completa está en el archivo `cases.md` del corpus `probability-monad`.

Los programas probabilísticos imprimen una salida canónica usando `Dist.norm`. Esta normalización agrupa resultados iguales y suma sus probabilidades, evitando que dos representaciones internas equivalentes se comparen como diferentes por el pipeline. La comparación sigue siendo byte a byte, pero sobre una representación normalizada de la distribución.

El corpus reutiliza varias familias estructurales de `Maybe`, como formas de dependencia, `binders`, `let`, `BodyStmt`, `shadowing`, selección por costo y controles. Además, incorpora familias específicas de probabilidades:

- distribuciones independientes con pesos no uniformes;
- resultados duplicados que deben sumarse al normalizar;
- soportes dependientes de valores generados previamente;
- pesos dependientes de `binders` anteriores;
- distribuciones construidas con `Dist.choose`, `Dist.relative` o `Dist.fromFreqs`;
- condicionamiento explícito mediante distribuciones imposibles o filtrado.

Esta separación permite validar dos dimensiones: que el `Renamer` se comporte igual que en el corpus estructural y que la semántica probabilística observable se conserve después del reordenamiento.

5.4.3. Control IO

El control con `IO` muestra por qué el reordenamiento no puede activarse para mónadas arbitrarias. En `IO`, dos acciones que no comparten variables pueden tener efectos observables cuyo orden importa. Por ejemplo, dos impresiones independientes pueden producir salidas distintas si se intercambian.

El log de validación del control negativo registra dos permutaciones. La permutación original imprime:

```
A
B
3
```

Una permutación alternativa imprime:

La comparación semántica detecta la diferencia y termina con **RESULT: FAIL**. Este resultado no invalida la solución; al contrario, confirma la necesidad del mecanismo de opt-in. El compilador no debe asumir conmutatividad por ausencia de dependencias RAW, porque los efectos observables pueden seguir dependiendo del orden.

Por esta razón, la extensión solo explora reordenamientos cuando el bloque fue marcado explícitamente como conmutativo mediante `CD.do`. Para bloques `do` normales, `ApplicativeDo` conserva el comportamiento estándar y no se habilita la enumeración de permutaciones conmutativas.

5.5. Resultados

Los resultados completos se obtienen desde los logs de resumen y de validación semántica generados para cada caso. En el estado actual del repositorio, los corpus sintéticos de `Maybe` y de `Dist.T Rational` están implementados y testeados; la tabla agregada final debe consolidar esos registros por familia y variante.

Como resultado representativo, el ejemplo académico probabilístico presentado a lo largo del informe reporta:

Caso	Sec.	ADo	Reord.	Perms.	Mínimas
Ejemplo académico	4	3	2	5	4

La tabla muestra la mejora estricta que motiva la memoria: el costo secuencial es 4, `ApplicativeDo` sobre el orden original reduce el costo a 3, y el reordenamiento conmutativo encuentra un plan de costo 2. Además, el caso genera cinco permutaciones válidas, de las cuales cuatro alcanzan el costo mínimo. El candidato 1, con orden `[1,3,2,4]`, es el primer mínimo y, por tanto, el seleccionado automáticamente.

La validación semántica ejecutó el programa original, `ApplicativeDo` sin reordenamiento, el candidato óptimo y cada una de las cinco permutaciones. Todas las variantes terminaron con código de salida cero y produjeron exactamente la misma distribución normalizada sobre preparación, quiz, dificultad y examen final. El registro `tests/main-example/semantic-validation.log` concluye con **RESULT: OK**.

Para el corpus sintético probabilístico, la tabla final debe reportar los mismos campos, junto con el resultado semántico de cada caso. La comparación debe realizarse sobre distribuciones normalizadas con `Dist.norm`. El control negativo con `IO` debe reportarse por separado, ya que su diferencia observable constituye el resultado esperado y no un fallo de los corpus evaluados bajo la hipótesis de conmutatividad.

5.6. Amenazas a la Validez

La evidencia experimental tiene varias amenazas a la validez que deben considerarse al interpretar los resultados.

En primer lugar, los dos corpus evaluados son sintéticos. Esto permite controlar formas de dependencia, patrones de binders, comportamiento probabilístico y casos de costo, pero limita la validez externa de los resultados: la frecuencia de las oportunidades de reordenamiento y la escalabilidad del espacio de búsqueda podrían ser distintas en programas de mayor tamaño o no diseñados para aislar estos fenómenos.

En segundo lugar, la validación semántica compara salidas observables. Esta estrategia es adecuada para los casos finitos evaluados, pero no constituye una prueba formal de preservación semántica para todo programa posible. En el corpus probabilístico, la normalización con `Dist.norm` reduce diferencias representacionales, pero también fija una convención de comparación que debe mantenerse explícita.

En tercer lugar, la enumeración de permutaciones puede crecer de forma combinatoria. Los casos pequeños permiten inspección exhaustiva, pero programas con muchos statements independientes podrían generar un espacio de búsqueda grande. Esta limitación afecta tanto el costo de compilación como la viabilidad de validar todos los candidatos.

Finalmente, la conmutatividad no es demostrada por `GHC`. El mecanismo de `CD.do` representa una afirmación del programador. Si se marca como conmutativo un bloque cuya mónada no preserva semántica bajo intercambio de efectos independientes, el compilador puede generar un programa observablemente distinto. El control con `IO` ilustra este riesgo.

También debe destacarse que el modelo de costos es uniforme por statement. Este modelo permite comparar planes de manera simple y reproducible, pero no distingue operaciones baratas de operaciones costosas. Un refinamiento de costos anotados o inferidos queda fuera de la implementación actual.

Capítulo 6

Conclusión

6.1. Resumen del Trabajo

Esta memoria presentó una extensión experimental de `ApplicativeDo` en `GHC` para explorar reordenamientos de statements en bloques `do` marcados como conmutativos. El punto de partida fue una limitación del algoritmo existente: `ApplicativeDo` puede descubrir independencia y construir planes applicative, pero preserva el orden relativo original de los statements. Esta decisión es correcta para mónadas arbitrarias, aunque puede ser conservadora cuando la mónada permite conmutar subcómputos independientes.

La solución implementada agrega un mecanismo explícito de opt-in sintáctico. Los bloques escritos como `CD.do` se reconocen usando un marcador exportado por el módulo conmutativo experimental. Para esos bloques, el Renamer construye un grafo de precedencia basado en dependencias RAW, enumera ordenamientos topológicos válidos, evalúa cada orden con el algoritmo existente de `ApplicativeDo` y selecciona el candidato de menor costo.

La validación se apoyó en corpus sintéticos para `Maybe` y para la mónada probabilística `Dist.T Rational`, además de un control negativo con `IO`. El ejemplo guía académico mostró una mejora estricta de costo: ejecución secuencial de costo 4, `ApplicativeDo` normal de costo 3 y reordenamiento conmutativo de costo 2. Sus cinco permutaciones conservaron exactamente la distribución normalizada del programa original y la validación terminó con `RESULT: OK`.

6.2. Cumplimiento de Objetivos

La Tabla siguiente resume el cumplimiento de los objetivos definidos en la Introducción.

Objetivo	Resultado
Analizar <code>ApplicativeDo</code> en <code>GHC</code> .	Se identificó el flujo del <code>Renamer</code> para <code>HsDo</code> , el punto donde se procesa <code>ApplicativeDo</code> y la construcción de <code>statements</code> aplicativos.
Diseñar un modelo de dependencias locales.	Se definió un grafo <code>RAW</code> usando variables definidas, variables libres y lecturas locales por <code>statement</code> .
Implementar la extensión experimental en <code>GHC</code> .	Se integraron el reconocimiento de la marca explícita, la generación de permutaciones topológicas, la evaluación de candidatos y la selección automática o forzada dentro del flujo de <code>ApplicativeDo</code> .
Evaluar la preservación de resultados y la reducción potencial del costo estructural.	Se probaron los corpus sintéticos de <code>Maybe</code> y <code>Dist.T Rational</code> con logs reproducibles de costos, grafos y salidas; además, el control negativo con <code>IO</code> mostró el riesgo de una declaración de conmutatividad incorrecta.

En conjunto, los objetivos quedaron cubiertos por la modificación de `GHC`, los módulos agregados, los dos corpus sintéticos, el control negativo con `IO`, los scripts de validación y la documentación del repositorio.

6.3. Limitaciones

La primera limitación es semántica: `GHC` no prueba que una mónada sea conmutativa. El mecanismo implementado se basa en una declaración explícita del programador mediante `CD.do`. Si esa declaración es incorrecta, el compilador puede aceptar reordenamientos que cambien efectos observables, como ilustra el control con `IO`.

La segunda limitación es combinatoria. Enumerar ordenamientos topológicos puede ser costoso cuando el bloque tiene muchos `statements` independientes. El prototipo permite estudiar exhaustivamente casos pequeños y medianos, pero requiere estrategias adicionales para escalar a programas de mayor tamaño.

La tercera limitación está en la validación. Comparar salidas observables entrega evidencia práctica para los casos evaluados, pero no equivale a una prueba formal de preservación semántica para todos los programas. En el corpus probabilístico, la normalización de distribuciones reduce diferencias representacionales, pero también fija una convención específica de comparación.

Finalmente, el modelo de costos es uniforme por `statement`. Esta simplificación permite comparar estructura de planes de manera reproducible, pero no distingue entre `statements` baratos y costosos. Por lo tanto, el menor costo abstracto no necesariamente equivale al menor tiempo de ejecución real.

6.4. Trabajo Futuro

Una línea directa de trabajo futuro es refinar el modelo de costos. La implementación actual usa costo uniforme por statement, siguiendo un criterio simple y reproducible. Una extensión posterior podría permitir costos anotados o inferidos, por ejemplo `low`, `medium` y `high`, interpretados como costos 1, 2 y 3. Con ello, dos planes con el mismo número de niveles applicative podrían diferenciarse por el peso relativo de sus statements.

```
defaultExample :: Maybe Int
defaultExample = CD.do
  x1 <- Just 5           -- high
  x2 <- Just 10          -- low
  x3 <- Just (x1 + 15)   -- medium
  x4 <- Just (x2 + 20)   -- high
  CD.return (x3 + x4)
```

Otra línea es reducir el espacio de búsqueda. En lugar de enumerar todos los ordenamientos topológicos, el compilador podría usar heurísticas, límites configurables o búsqueda guiada por costo parcial. Esto permitiría aplicar la idea a bloques más grandes sin pagar siempre el costo de una enumeración exhaustiva.

También queda abierta una evaluación de rendimiento real. La memoria compara planes usando un costo abstracto heredado de `ApplicativeDo`; un trabajo posterior podría medir tiempos de compilación, tiempos de ejecución y efectos sobre backends o librerías que aprovechen paralelismo applicative.

Finalmente, sería valioso integrar casos de esta extensión en una testsuite más cercana a `GHC`, mejorar las trazas para inspección humana y avanzar hacia una formalización más completa de la preservación semántica bajo la hipótesis de conmutatividad.

6.5. Cierre

El trabajo muestra una vía concreta para ampliar `ApplicativeDo` sin abandonar el compilador real ni la sintaxis habitual de `Haskell`. Bajo una hipótesis explícita de conmutatividad, el orden escrito por el programador deja de ser el único orden que el compilador puede evaluar, siempre que se preserven las dependencias locales de variables.

El aporte principal no es solo el algoritmo de reordenamiento, sino su integración experimental en `GHC` con trazabilidad: módulos agregados, parches, corpus, scripts y logs reproducibles. Esto permite estudiar la idea de manera concreta, observar sus beneficios en casos controlados y delimitar sus riesgos.

En síntesis, la memoria entrega evidencia experimental de que el espacio de planes de `ApplicativeDo` puede ampliarse para mónadas declaradas como conmutativas, obteniendo mejoras de costo sin modificar la semántica observable en los casos sintéticos evaluados.

Bibliografía

- [1] Esteban Romero Berríos. Repositorio experimental para el estudio de applicative-do y reordenamiento de statements en ghc, 2026. GitHub repository.
- [2] Alan L. Davis and Robert M. Keller. Data flow program graphs. *Computer*, 15(2):26–41, 1982.
- [3] Martin Erwig and Steve Kollmansberger. Functional pearls: Probabilistic functional programming in haskell. *Journal of Functional Programming*, 16(1):21–34, 2006.
- [4] Martin Erwig and Steve Kollmansberger. *Numeric.Probability.Distribution: Deterministic and Probabilistic Values*. Hackage, 2026. Documentación del paquete `probability`, versión 0.2.9.1.
- [5] Kai Feng, Jeremy Singer, and Angelos K. Marnierides. Fuzzrducc: Fuzzing with reconstructed def-use chain coverage, 2025.
- [6] Zoubin Ghahramani. Probabilistic machine learning and artificial intelligence. *Nature*, 521(7553):452–459, 2015.
- [7] GHC Contributors. Commit 8ecf6d8f7dfce9e5b1844cd196f83f00f3b6b879, 2015. GitHub commit.
- [8] GHC Contributors. Testsuite de la extensión applicativo en ghc, 2026. Directorio `testsuite/tests/ado` del repositorio oficial de GHC.
- [9] The Glasgow Haskell Compiler Team. *GHC User’s Guide: Applicative do-notation*, 2026. Documentación oficial de GHC.
- [10] The Glasgow Haskell Compiler Team. *GHC User’s Guide: Qualified do-notation*, 2026. Documentación oficial de GHC.
- [11] Andrew D. Gordon, Thomas A. Henzinger, Aditya V. Nori, and Sriram K. Rajamani. Probabilistic programming. In *Future of Software Engineering Proceedings*, pages 167–181. ACM, 2014.
- [12] Mary Jean Harrold and Mary Lou Soffa. Efficient computation of interprocedural definition-use chains. *ACM Transactions on Programming Languages and Systems*, 16(2):175–204, 1994.
- [13] Bart Jacobs. From probability monads to commutative effectuses. *Journal of Logical and Algebraic Methods in Programming*, 94:200–237, 2018.

- [14] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.
- [15] Anders Kock. Commutative monads as a theory of distributions. *Theory and Applications of Categories*, 26(4):97–131, 2012.
- [16] Simon Marlow. *Haskell 2010 Language Report*. Haskell Community, 2010. Informe oficial de la especificación del lenguaje.
- [17] Simon Marlow. `ado`, 2015. GitHub repository.
- [18] Simon Marlow, Louis Brandy, Jonathan Coens, and Jon Purdy. There is no fork: An abstraction for efficient, concurrent, and concise data access. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, pages 325–337. ACM, 2014.
- [19] Simon Marlow, Simon Peyton Jones, Edward Kmett, and Andrey Mokhov. Desugaring Haskell’s `do`-notation into applicative operations. In *Proceedings of the 9th International Symposium on Haskell*, pages 92–104. ACM, 2016.
- [20] Simon Marlow, Ryan Newton, and Simon Peyton Jones. A monad for deterministic parallelism. In *Proceedings of the 4th ACM SIGPLAN Symposium on Haskell*, pages 71–82. ACM, 2011.
- [21] Conor McBride and Ross Paterson. Applicative programming with effects. *Journal of Functional Programming*, 18(1):1–13, 2008.
- [22] Andrey Mokhov, Georgy Lukyanov, Simon Marlow, and Jeremie Dimino. Selective applicative functors. *Proceedings of the ACM on Programming Languages*, 3(ICFP):1–29, 2019.
- [23] Justin Pombrio. *Resugaring: Lifting Evaluation Sequences through Syntactic Sugar*. PhD thesis, Brown University, 2018.
- [24] Adam Ścibior, Zoubin Ghahramani, and Andrew D. Gordon. Practical probabilistic programming with monads. In *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell*, pages 165–176. ACM, 2015.
- [25] The Glasgow Haskell Compiler Team. `GHC.Internal.Base`: Basic definitions. Código fuente oficial de GHC, 2026. Commit `0b36e96`, <https://gitlab.haskell.org/ghc/ghc/-/blob/0b36e96cb93db71f201aaa055c4a90b75a8110ef/libraries/ghc-internal/src/GHC/Internal/Base.hs>.
- [26] Philip Wadler. Monads for functional programming. In *Advanced Functional Programming*, volume 925 of *Lecture Notes in Computer Science*, pages 24–52. Springer, 1995.
- [27] Brent Yorgey. The typeclassopedia. *The Monad.Reader*, 13:17–68, 2009.

Apéndice A

Anexos Técnicos

A.1. Reproducibilidad

Esta sección de anexo debe documentar el entorno reproducible. Existe para que el cuerpo principal no se sobrecargue con detalles de Dev Container, toolchain y build de GHC. Lo ideal es incluir comandos, versiones y referencias a `docs/toolchain.md`.

Para completarla, se deben listar versiones de GHC de bootstrap, Cabal, Alex, Happy y comandos Makefile relevantes. También conviene explicar la ruta esperada del binario modificado de GHC dentro del contenedor.

A.2. Pipeline de Validación

Esta sección de anexo debe describir con detalle los comandos usados para validar. Existe para que el capítulo Validación pueda concentrarse en metodología y resultados, dejando aquí la información operativa. Lo ideal es incluir ejemplos de `make test-ghc` y `make test-cabal`.

Para completarla, se debe documentar la estructura de entrada bajo `experiments/` y la estructura de salida bajo `tests/`. También conviene explicar la diferencia entre backend GHC directo y backend Cabal para dependencias externas como `probability`. El ejemplo guía debe presentarse como un recorrido completo del backend Cabal, desde `experiments/main-example/main.hs` hasta los resultados de `tests/main-example`.

A.3. Logs y Trazas

Esta sección de anexo debe reunir ejemplos de logs representativos. Existe porque las trazas completas del Renamer son demasiado densas para el cuerpo principal, pero son importantes para reproducibilidad y depuración. Lo ideal es incluir fragmentos de `summary.log`,

`optimal-reorder.log` y `semantic-validation.log`.

Para completarla, la fuente principal debe ser `tests/main-example: summary.log` resume los costos, `graph.log` presenta las dependencias, `logs/original_ado.log` muestra el plan sin reordenamiento, `logs/optimal-reorder.log` registra el candidato elegido y `semantic-validation.log` confirma la preservación de la salida. También conviene incluir un caso estructural de `Maybe` y el control negativo con `IO` para mostrar comportamientos complementarios.

A.4. Corpus

Esta sección de anexo debe contener tablas completas de los corpus. Existe para que Validación pueda presentar solo casos representativos sin perder trazabilidad del conjunto completo. Lo ideal es basarse en `experiments/maybe-monad/cases.md` y `experiments/probability-monad/ca`

Para completarla, se deben copiar o sintetizar las tablas de familias, casos y variantes implementadas para ambos corpus sintéticos. El control negativo con `IO` debe documentarse por separado para distinguirlo de los casos evaluados bajo la hipótesis de conmutatividad.

A.5. Detalles de Implementación

Esta sección de anexo debe contener los detalles de GHC que serían demasiado densos para el capítulo Solución. Existe para preservar trazabilidad técnica sin interrumpir la lectura principal. Lo ideal es referenciar `docs/renamer.md` y los parches relevantes.

Para completarla, se pueden listar funciones como `rearrangeForApplicativeDo`, `isCommutativeQua`, `buildStmtsDependencyGraph` y `enumerateSemanticTopSortsBounded`. También conviene mencionar los módulos agregados en `base` y `ghc-internal`.